SE(3)-WAM: Role-Aware 4D World–Action Modeling for Dynamic Manipulation

Anonymous CVPR submission

Figure placeholder: Fig. 1 teaser — a role-aware 4D view of a moving scene (real robot)

Layout: full width, two parts. **(a) Left, ∼62%:** one real-robot frame of the *linear-motion grasp* at the instant a chunk is issued. Overlays in the paper's semantic colors: blue = the target's past SE(3) poses from the causal tracker (small triads at $-350 \ldots 0$ ms, fading with age); orange = the target's SE(3) forecast at the execution times of the chunk, taken from the training-time heads evaluated offline; light orange arrows = sparse 3D flow on the 14×14 grid over the whole scene; purple = the commanded gripper path meeting the target at τ^{a}. A gray ghost marks the pose observed at capture, with a dashed red arrow showing where a reactive policy would close. A thin timeline strip under the image: capture $\tau_t \rightarrow$ latency $\ell_t \rightarrow$ execution τ^{a}. **(b) Right, ∼38%:** top: "Image-space WAM: future frames or latents at uniform fidelity" (filmstrip icon); middle: "SE(3)-WAM: SE(3) for the target, sparse 3D flow for the whole scene, training only" (triads + sparse arrows); bottom **(required)**: SR vs. added latency (0–200 ms) for SE(3)-WAM and the strongest baseline (data: the latency sweep of Appendix C.2). **Rules:** real photo kept unretouched; overlays drawn from logged tracker/forecast/command data of that trial; caption states the trial's outcome.

Figure 1. **A role-aware 4D view of a moving scene.** Between image capture and the first command, a part on the conveyor moves farther than the grasp tolerates. SE(3)-WAM encodes the target's metric motion (blue) and is trained to forecast the target's SE(3) pose precisely and the whole scene coarsely as a sparse 3D flow (orange), so that its commands meet the target when they execute (purple). Image-space world–action models forecast appearance at uniform fidelity instead. Right: success against added latency on Dynamic-14.

Abstract

Manipulating moving objects requires actions that fit the scene when they execute, not when the image was captured, yet every millisecond spent inferring the scene's motion lets the target move further. World–action models forecast future images or latents at uniform fidelity, spending time on appearance while leaving the target's metric pose implicit; target-centric predictors are fast but ignore the surroundings that constrain or drive the motion. We observe that the parts of a dynamic scene need different precision: the target's pose decides contact to within millimeters, the bodies around it matter only through the clearance they leave and the motion they impart to the target, and the remaining background does not matter. We present SE(3)-WAM, a world–action model built on this role-aware 4D structure. It encodes the target's motion as a metric SE(3) pose history and is trained to forecast the target's future SE(3) trajectory precisely and the whole scene coarsely as a sparse 3D flow. The forecasts serve only as training supervision and are skipped at deployment, so inference needs neither past frames nor a generated future and stays fast enough for real-time control. On Dynamic-14, a simulated benchmark in which the scene keeps moving during inference, SE(3)-WAM achieves [XX.X]% success, [XX.X] points above the strongest baseline, and succeeds in [XX]% of trials on six real-robot tasks with diverse target motion.

1. Introduction

Learning-based manipulation has advanced rapidly, from diffusion policies to vision–language–action (VLA) and world–action models (WAMs) [5, 10, 22, 35, 49, 56, 71], yet most of this progress assumes that objects stay still unless the robot moves them. In factories, kitchens, and human–robot handovers, objects keep moving while the robot perceives, computes, and acts. This poses two coupled difficulties. A single image does not reveal how the target is moving, and by the time an action executes, the target is no longer where it was seen: a part on a conveyor at 0.1 m/s travels 15 mm during a 150 ms delay, more than a parallel-jaw grasp typically tolerates (Fig. 1).

Recent work equips policies with a memory of past motion and a forecast of the future [12, 31, 44, 63]. Both cost the resource that dynamic manipulation lacks most, time: richer histories and denser forecasts add computation, and the added latency enlarges the very error the forecast is meant to remove. Existing designs therefore trade information against speed. Video-based WAMs forecast whole future frames or latents at uniform fidelity [49, 56], also when conditioned on motion cues, which remain in image space and do not separate the target from other moving regions [31], and methods that restrict prediction to dynamic regions or salient tokens still predict in image space [44, 63]. In both, most capacity goes to appearance, and the quantity that decides contact, the target's metric pose when the ac-

tion executes, stays implicit. Target-centric predictors and dynamic VLAs with object-level forecasts sit at the other end [12, 17, 32]: they are fast but blind to surroundings that can block or redirect the motion. None of them provides knowledge of the moving scene that is precise where contact depends on it, coarse elsewhere, and cheap to use at execution time.

Our key observation is that a dynamic policy needs to know very little precisely. Contact depends on the target's pose at the moment the action executes, to within millimeters. The bodies around it matter only through the clearance they leave and the motion they impart to the target, which a coarse estimate captures, and the rest of the background does not matter. Nor does this knowledge have to be computed at test time: a forecast that supervises training shapes what the policy represents and can be dropped at deployment [60]. SE(3)-WAM builds on both ideas (Fig. 2). It sees the current image together with short metric SE(3) histories of the target and the robot, which convey motion directly instead of a stack of past frames. A world stream is trained to forecast, in one metric 4D space, the target's future SE(3) trajectory precisely and the whole scene coarsely as a sparse 3D flow, and an action stream reads the world stream's summary and replans as often as its latency permits. SE(3)-WAM thus gains the precision of a metric forecast at a latency close to that of a reactive policy.

We evaluate SE(3)-WAM on Dynamic-14, a simulated benchmark of fourteen tasks in which observation, scene motion, and action execution share one physical clock, so the scene keeps moving while a policy computes. SE(3)-WAM achieves state-of-the-art success of [XX.X]%, [XX.X] points above the strongest baseline, at a p95 latency of [XX] ms, and succeeds in [XX]% of trials on six real-robot tasks, against [XX]% for the strongest baseline; it also leads on the 24 tasks of DOMINO [12]. Five final-scale controls isolate the contributions of target-motion history, future supervision, and its two geometric components, and compare role-aware with uniform-fidelity forecasts.

Our contributions are threefold:

- We propose role-aware 4D modeling for dynamic manipulation. Tracing how latency and estimation errors reach the contact shows that only the target needs millimeter precision, so we represent the target precisely by its metric SE(3) pose history and future trajectory, and the whole scene coarsely by a sparse 3D flow, in metric 3D rather than in image space.
- We present SE(3)-WAM, a world–action model trained with these role-aware 4D forecasts as supervision, at a latency close to that of a reactive policy.
- We build Dynamic-14, in which the scene keeps moving during inference, and show state-of-the-art results on it, on DOMINO, and on six real-robot tasks.

2. Related Work

Dynamic manipulation. Compensating for the time between sensing and acting is a classical problem, from control with dead time [41] to catching systems that track the target with Kalman-type filters and plan an interception [1–3, 34]; such systems are precise but engineered for each object and task. Learned policies have recently been brought to moving scenes. PUMA reads historical optical flow and is supervised with future object features [12], DynamicWAM conditions a WAM on flow and kinematic descriptors and executes it with real-time chunking [31], and other methods forecast salient tokens, future images, or compact future codes and feed them to the policy at test time [32, 44, 61, 64]; DynaWM refines the chunks of a base VLA for moving objects [9], and DynamicManip synthesizes dynamic demonstrations from a static one [26]. Execution schemes instead reduce or hide latency, by overlapping inference with execution [37, 39, 54], keeping consecutive chunks consistent [6, 29], or conditioning on the execution time [45, 48]. They decide when actions are computed and applied, whereas SE(3)-WAM changes what the policy represents; we run every method under the same real-time executor and, as a control, with its original executor. DOMINO [12] and PhysMani-Bench [61] provide diverse dynamic-manipulation tasks, but their released default evaluation pipelines advance the simulation only after a policy has returned its prediction; Dynamic-14 advances the scene during inference and charges each method its measured latency.

Predictive supervision in world–action models. World–action models (WAMs) generate future video and actions in one network [25, 49, 56, 69], building on video prediction as a policy objective [18, 46, 53]. Because video is costly and dominated by appearance, recent work changes what is forecast, such as dynamic regions, depth, and semantic features [63], flow and geometry alongside RGB [59], object masks or slots [30, 57], or compact latents [23, 24], and asks whether the forecast is needed at test time: Fast-WAM retains most of the benefit of video modeling without test-time imagination [60], and PUMA and others drop their predictors after training [12, 43, 65]. SE(3)-WAM adopts this training-only use and changes the target of the supervision: instead of pixels, features, or latents at one resolution, a metric 4D state whose precision follows each entity's role in contact.

Geometric representations of motion. Metric 3D inputs make policies more precise and sample-efficient [14, 20, 38, 62]. Motion has been represented by 2D point tracks [4, 51, 55, 67], 3D flow [15, 19, 58, 68], and object keypoints or SE(3) trajectories [17, 33, 42, 52], and rigid-body decompositions have long been used to learn scene dynamics [7, 8]. These works describe the scene with one kind of

representation, dense flow or tracks everywhere or a pose for each object; DOCIR separates the robot, the target, and obstacles, but as input encodings [11]. SE(3)-WAM assigns precision by role: a coarse sparse 3D flow describes the whole scene, and the target, on which contact depends to within millimeters, is additionally described by its SE(3) history and forecast.

3. SE(3)-WAM

SE(3)-WAM is a world–action model that represents a dynamic scene at role-dependent precision (Fig. 2). It encodes the current RGB-D frame, whose RGB a frozen MAE ViT-B/16 [16] turns into 196 patch tokens and whose depth gives each patch the metric 3D position of its center point in the robot base frame, together with short histories of the target's SE(3) pose and of the robot state. These 212 scene tokens and 16 learnable summary tokens that aggregate them form a world stream, which forecast heads supervise during training with the target's future SE(3) trajectory and a sparse 3D flow of the whole scene. An action stream shares attention with the world stream, reads its summary tokens, and generates action chunks by conditional flow matching [27].

3.1. Problem Formulation

SE(3)-WAM learns a policy from demonstrations. At query t, the policy receives an RGB image, a depth map, and the robot state captured at time τ_t, together with short histories of the robot state and of the pose $T(\tau) = (R(\tau), \mathbf{p}(\tau)) \in \mathrm{SE}(3)$ of a target object in the robot base frame. It outputs a chunk $(\mathbf{a}_{t,0}, \ldots, \mathbf{a}_{t,H-1})$ of commands, one per control period Δ_a, indexed by time from capture: the j-th command is meant for

$$\tau^{\mathrm{a}}_{t,j} = \tau_t + j\Delta_a + d_{\mathrm{task}}. \tag{1}$$

Here d_{task} is a task's built-in command delay. The chunk becomes available only after a latency ℓ_t, so its commands with $j\Delta_a < \ell_t$ have expired, and the later ones execute at their times $\tau^{\mathrm{a}}_{t,j}$. A command must therefore fit the target's pose at $\tau^{\mathrm{a}}_{t,j}$ rather than at τ_t, a pose that the policy must estimate from observations at τ_t, and the lead $\Delta = \tau^{\mathrm{a}}_{t,j} - \tau_t$ of every executed command is at least ℓ_t, so it grows with every millisecond of inference. The bodies of a scene play different roles. The *target* is the body whose pose must be matched within a contact or containment tolerance ε, such as an object to grasp or a receptacle to place into. The *context* consists of the bodies that constrain or drive the target's motion: obstacles, such as an occluder or a wire, must be avoided with a clearance m, and drivers, such as a conveyor, a ramp, or a striker, move the target. The rest of the scene does not affect the task. Sec. 3.2 traces how the error of this estimate depends on the lead and on these roles, and derives the representation of SE(3)-WAM from it.

3.2. Role-Aware 4D Representation

What must be precise. Let a policy act, explicitly or implicitly, on an estimate of the target pose at τ^{a} with position error e and rotation error θ. A contact point at lever arm r is then off by at most $e + 2r\sin(\theta/2)$. Writing the target's motion over the lead as a constant velocity $\mathbf{v}$ plus a residual (acceleration, rolling, impacts), and the lead that the policy assumes as $\hat{\Delta}$, gives

$$e \le e_{\mathrm{loc}} + \hat{\Delta}\, e_v + \|\mathbf{v}\|\, e_\tau + e_{\mathrm{dyn}}, \tag{2}$$

with the errors e_{loc} of the current position, e_v of the velocity, $e_\tau = |\hat{\Delta} - \Delta|$ of the timing, and e_{dyn} of the residual motion (Appendix A.1). The tolerance ε is typically a few millimeters, and the rotation error is amplified by the lever arm, so the target's pose and motion must be known precisely. The context needs far less precision. An obstacle only has to be avoided with the clearance m, which is usually far larger than ε, and a driver matters through the motion it imparts to the target, which then appears in the target's own future; in both cases, the policy needs to know only roughly where the context moves (Fig. 3c). We therefore represent the target by metric SE(3) states, both its observed history and its forecast trajectory, and the whole scene by a sparse 3D flow, all in the robot base frame and in physical time.

What latency allows. Latency enters Eq. (2) twice, through the lead that multiplies every velocity error and through the motion that the residual accumulates. Information that costs inference time therefore pays off only if it removes more error than the added latency creates. SE(3)-WAM keeps its inputs cheap, with one current image and low-dimensional state histories instead of past frames, and uses its forecasts only to supervise training, so that they add no inference time (Sec. 3.4).

Target motion: history and forecast. A single pose does not reveal how the target moves (Fig. 3a). The target history covers the last 8 control steps (350 ms at 20 Hz). It consists of the current target pose in the base frame and the seven interval motions between consecutive samples, each with its translation, relative rotation, time interval, and finite-difference velocity and acceleration in physical units, and forms 8 target-state tokens (Appendix A.2). These tokens state directly the velocity and acceleration on which e_v and part of e_{dyn} depend, at the cost of a few low-dimensional tokens. The robot history adds 8 end-effector state tokens. In simulation, the target pose comes from the simulator; on the real robot, a causal tracker estimates it from the RGB-D stream (Appendix D).

The forecast describes where the target will be when the commands execute (Fig. 3b). It covers fixed physical times $\tau_t + \delta_h$, with $\delta_h = h{\cdot}50\,\mathrm{ms}$ and $h = 1, \ldots, 8$, from 50 ms to

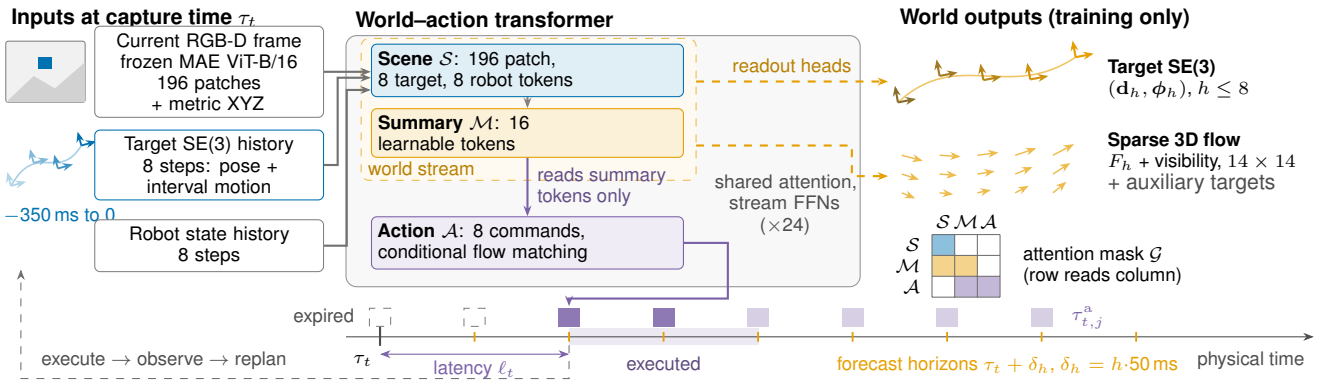


Figure 2. **SE(3)-WAM.** The world stream holds 212 scene tokens, the current image patches from a frozen encoder together with short histories of the target's metric SE(3) pose and of the robot state, and 16 summary tokens. During training, light heads read the world stream out into a role-aware 4D forecast at physical horizons $\tau_t + \delta_h$: the target's SE(3) trajectory and a sparse 3D flow with visibility on a 14×14 grid over the whole scene (dashed: training only). The action stream generates a chunk of 8 commands by flow matching. The attention mask $\mathcal{G}$ lets scene tokens read the scene, summary tokens read the scene and summaries, and actions read summaries and actions. The policy observes and replans at the first control boundary at or after each query ends, where the new chunk takes over from the command meant for that boundary.

Figure placeholder: Fig. 3 — acting on a moving target (schematic)

Layout: one row, three panels (a)–(c) of equal width, clean vector schematic in the paper's semantic colors; the same robot arm and T-shaped part in all panels; no photographs. **(a) Same pose, different motion:** two histories of the part, A (solid blue) and B (dashed blue), with ghost poses at -100, -50, 0 ms that end in the same pose at capture τ_t. Label: "current pose alone is ambiguous". **(b) Pose at execution:** a timeline strip under the panel: capture τ_t, then ticks every $\Delta_a = 50$ ms at the times $\tau^{\mathrm{a}}_{t,j}$ of the chunk's commands; the ticks before the chunk arrives at $\tau_t + \ell_t$ are grayed out as expired, and the first executed command is marked τ^{a}. Above it, the part at τ^{a} for A and B with different positions and rotations (orange triads). Label: "the command must fit the pose at τ^{a}". **(c) Roles:** one scene with three kinds of bodies. *Target*: the part with the end-effector frame E, the target frame O, the contact point, and tolerance ε (mm). *Context, obstacle*: a receptacle wall or occluder with a clearance arrow $m \gg \varepsilon$. *Context, driver*: the conveyor carrying the part, with a motion arrow, and optionally a striker approaching it; label "moves the target". *Background*: table edge and distant objects desaturated in gray; label "no effect". A three-row legend: target → precise SE(3); context → coarse 3D flow; background → nothing. **Rules:** conceptual illustration with hypothetical positions; blue past, orange future, purple command, gray background.

Figure 3. **Acting on a moving target** (schematic). (a) Two histories reach the same pose at capture time τ_t but imply different motions. (b) At the execution time τ^{a}, they lead to different target poses. (c) A contact requires the end-effector frame E to match the target O within ε; an obstacle in the context matters only through the gap $m \gg \varepsilon$ it leaves, a driver such as the conveyor matters through the motion it imparts to the target, and the background does not affect the task.

400 ms after capture, so that every command time $\tau^{\mathrm{a}}_{t,j}$ with $j \ge 1$ is a forecast horizon. Like all our motion labels, it is cumulative from the current state, and every loss averages over valid labels only. For each horizon, the target's motion is

$$\mathbf{d}_h = \mathbf{p}(\tau_t + \delta_h) - \mathbf{p}(\tau_t), \quad \phi_h = \mathrm{Log}\big(R(\tau_t + \delta_h) R(\tau_t)^\top\big)^\vee. \tag{3}$$

A linear pose head on the mean of the final summary tokens predicts $(\hat{\mathbf{d}}_h, \hat{\phi}_h)$ with the loss

$$\mathcal{L}_P = \frac{1}{|\mathcal{V}|} \sum_{h \in \mathcal{V}} \Big[\|\hat{\mathbf{d}}_h - \mathbf{d}_h\|^2 + \lambda_R \, d_{\mathrm{SO}(3)}\big(\hat{R}_h, R_h\big)^2 \Big], \tag{4}$$

where $\hat{R}_h = \mathrm{Exp}(\hat{\phi}_h^\wedge) R(\tau_t)$ and $\mathcal{V}$ is the set of valid horizons. Because the pose head reads the summary tokens, the only world tokens that the actions attend to (Sec. 3.3), the precise target supervision shapes exactly the representation that the actions use. The past and the future of the target thus share one metric frame and one time axis.

Scene motion: sparse 3D flow. The center points of the 196 patches on the 14×14 grid cover the target, the robot, and the surroundings. For each point i and horizon h, a linear head on its final patch token predicts the base-frame displacement $\hat{F}_{ih} \in \mathbb{R}^3$ and a visibility logit $\hat{o}_{ih}$, and

$$\mathcal{L}_F = \frac{1}{|\mathcal{V}_F|} \sum_{(i,h) \in \mathcal{V}_F} \Big[\tfrac{1}{3} \big\| (\hat{F}_{ih} - F_{ih})/s \big\|^2 + \mathrm{BCE}(\hat{o}_{ih}, o_{ih}) \Big], \tag{5}$$

with a fixed scale s fitted on the training labels, the visibility o_{ih}, and the set $\mathcal{V}_F$ of valid labels. The grid samples the workspace sparsely, about [X] mm between points. The flow thus gives the context and the robot a motion estimate at this spacing and stays near zero on the static background, while the target additionally receives the dedicated SE(3) supervision of Eq. (4). The flow supervises the patch tokens from which the summaries draw, one level below the precise target supervision; this is how SE(3)-WAM allocates precision by role.

Auxiliary supervision. Two further targets regularize the representation; we selected them on development scenes (Appendix E). Future appearance heads on the patch tokens predict, at every horizon, the metric depth, the features of a frozen DINOv3 teacher [40], and the RGB of moving surfaces. Separately, a small head on the summary tokens reconstructs the RGB of the current target region, which ties the summaries to the target without a future label. Their losses form $\mathcal{L}_{\text{aux}}$ (Appendix A.3).

3.3. Action Generation and Joint Learning

Action stream. The action stream holds $H = 8$ tokens, one per command of the chunk; each embeds a noisy command, an end-effector translation and rotation increment plus a gripper command (Appendix A.4). The attention mask $\mathcal{G}$ lets scene tokens attend to the scene, summary tokens to the scene and the summaries, and action tokens to the summaries and the actions, so the actions read the world through the summary tokens and the world stream does not depend on the actions. In every layer, both streams share the attention projections, through which they exchange information, and keep separate feed-forward networks, which give each its own capacity; we selected this backbone over fully separate parameters on development scenes (Appendix E.1). World tokens use layer normalization, and action tokens use adaptive layer normalization conditioned on the flow time.

Joint learning. For a chunk $\mathbf{a}_1$ normalized with training statistics (Appendix A.4), noise $\mathbf{a}_0 \sim \mathcal{N}(0, I)$, and flow time $u \sim \mathcal{U}[0, 1]$, the action stream v_ψ, with parameters ψ, regresses the flow-matching velocity

$$\mathcal{L}_A = \mathbb{E} \left\| v_\psi(\mathbf{a}_u, u \mid \mathcal{S}_t, \mathcal{G}) - (\mathbf{a}_1 - \mathbf{a}_0) \right\|^2, \qquad (6)$$

with $\mathbf{a}_u = (1 - u)\mathbf{a}_0 + u\mathbf{a}_1$, the scene tokens $\mathcal{S}_t$ of query t, and the error averaged over valid commands. All parts are trained jointly with

$$\mathcal{L} = \mathcal{L}_A + \lambda_P \mathcal{L}_P + \lambda_F \mathcal{L}_F + \lambda_{\text{aux}} \mathcal{L}_{\text{aux}}, \qquad (7)$$

with the weights given in Sec. 4.1.

3.4. Efficient Inference and Execution

SE(3)-WAM trains its forecast heads jointly and skips them at test time, as recent WAMs do [25, 60]. Because world tokens attend neither to the actions nor to the flow time, the world stream runs once per query, and the action stream integrates 5 Euler steps on its cached keys and values. The executor counts latency in whole control periods, as RTC does [6]. A query of total latency L_t hands its chunk over at the first control-period boundary at or after it ends, $\ell_t = k_t \Delta_a$ after capture with $k_t = \lceil L_t / \Delta_a \rceil$, and execution continues from command k_t, the one meant for that boundary. There the policy observes again and starts the next query; until then, the executor continues the previous chunk. The policy therefore replans as often as its latency allows, before every command when a query fits in one control period. Because each command is issued at the time its label refers to, the action stream needs no latency input, and the timing error e_τ of Eq. (2) is limited to actuation delays. Every executed command, at most $(H-1)\Delta_a + d_{\text{task}}$ after capture, lies within the horizons of the forecast supervision (Appendix B.3). On the robot, L_t also includes tracking the target in the newest frame.

4. Experiments

We evaluate how SE(3)-WAM performs when the scene keeps moving during inference (Sec. 4.2), what each of its components contributes (Sec. 4.3), and how it transfers to a real robot (Sec. 4.4).

4.1. Setup

Dynamic-14. Dynamic-14 is a simulated benchmark in which observation, scene motion, and action execution share one physical clock, so the scene keeps moving while a policy computes and every method pays for its own latency. It contains fourteen tasks in MuJoCo/robosuite [47, 70] in five families: grasping a moving target (harmonic, conveyor, translating, rotating, and occluded motion), placing into a moving receptacle, passing a wand along a rotating wire, capturing rolling or sliding objects, and grasping after an impact (Fig. 4; Appendix B). Experts with privileged simulator state provide 800 demonstrations per task. We select configurations on development scenes and report all Dynamic-14 results on 1,400 held-out scenes (100 per task, new seeds).

Evaluation. During each query, physics advances by the method's own inference latency, drawn from its latency distribution measured at batch size 1 on one RTX 5090, the GPU of our robot, independently of the hardware that runs the rollouts (Appendix B.3). All methods share one executor that indexes each chunk by time from capture (Sec. 3.4): a query starts at a control-period boundary, and its chunk takes over at the first boundary at or after the query ends (Appendix B.3). Besides this native-latency setting, we add 150 ms of latency to every method and pause physics during inference; Appendix C.2 adds equal-latency and original-executor controls. We report the success rate (SR), averaged over the fourteen tasks with equal weight and over training seeds, and the p95 latency, the measured time from an observation to its first command (time-to-action). Paired 95% intervals come from a bootstrap over held-out scenes.

Baselines. We compare with reactive policies, dynamic VLAs, image- or latent-space WAMs, and a dynamic WAM

Figure placeholder: Fig. 4 — the fourteen Dynamic-14 tasks

Layout: full width, two rows of seven equal panels, one per task, rendered agent-view frames of Dynamic-14 (no photographs). A thin family label bar spans each group of panels. Row 1: *Grasp* (harmonic, conveyor, translating, rotating, occluded) and *Place* (moving receptacle, rotating receptacle). Row 2: *Pass* (wire passage, full wire passage), *Capture* (ball, ball on table, slider, can), and *Impact* (post-impact grasp). Task name under each panel. **Each panel:** cropped to the region in which the target moves, at the same scale in all panels; 3–4 semi-transparent past poses of the target and an arrow for its motion, so that the motion pattern is visible (oscillation, straight line, rotation, disappearance behind the occluder, rolling, deflection after the impact); obstacles outlined and drivers marked in the role colors of Fig. 3c. **Rules:** no timeline (Figs. 2 and 3 already show one); same camera in all panels.

Figure 4. **Dynamic-14.** Fourteen tasks in five families, each shown with the recent motion of its target. Observation, scene motion, and execution share one physical clock, so the scene keeps moving while a policy computes.

(Table 1). Each baseline is trained on the same demonstrations with its official code, inputs, and recipe, including a rendered wrist camera and frame or flow histories where the recipe uses them (Appendix B.4). SE(3)-WAM uses one agent view with its depth and the target's SE(3) history, without past frames. To give baselines the same target-motion information, $\pi_{0.5}$ and DynamicWAM also run with our SE(3) history tokens (+SE(3)).

Implementation. The transformer has 24 layers of width 1024 with 16 heads and 523.8M trainable parameters; the MAE ViT-B/16 encoder and the DINOv3 teacher stay frozen. On Dynamic-14, it trains from random initialization for 20 epochs with AdamW, $\lambda_P = \lambda_F = 1$, and $\lambda_R = \lambda_{\text{aux}} = 0.1$ (Appendix A.5). On the real robot, it is fine-tuned per task from a 48-layer base pretrained on DROID [21] (Appendix A.6).

4.2. Simulation Results

Comparison. With its native latency, SE(3)-WAM reaches [XX.X]% SR on held-out Dynamic-14 scenes (Table 1), [X.X] points above the strongest baseline, [name], and [X.X] points above the strongest baseline that receives the same SE(3) history, [name]. Its gains are largest on [task types] (Appendix C.1). Its p95 latency of [XX] ms is [X]× lower than that of the image-space WAMs and close to that of [the reference reactive policy, to be fixed].

Latency and protocols. With 150 ms of added latency, SE(3)-WAM keeps [XX.X]% SR, whereas the reactive policies drop to [XX.X]% and the image-space WAMs to [XX.X]% on average, as Eq. (2) predicts: added latency lengthens the lead that multiplies every velocity error, so methods that do not anticipate the target's motion lose most. Fig. 1 (right) and Appendix C.2 give the full sweep from 0 to 200 ms. With physics paused during inference, which ignores the cost of computation, SE(3)-WAM still leads [name] by [X.X] points, against [X.X] points with native latency. When every method is charged the p95 latency of SE(3)-WAM, its margin is [X.X] points, and running [the baselines that rely on their own asynchronous execution]

Table 1. **Main results** on 1,400 held-out Dynamic-14 scenes. SR (%) averages the fourteen tasks with equal weight and three training seeds for SE(3)-WAM (one for baselines), with each method's native latency, with 150 ms of added latency, and with physics paused during inference. p95 lat.: p95 latency, the time-to-action (ms) at batch size 1 on one RTX 5090; these latencies are replayed in simulation; rollout hardware is reported in Appendix C.2. All methods receive the current agent-view RGB and the robot state. Wrist: an additional wrist camera. History: past RGB frames (RGB), an optical-flow or flow-based motion history (Flow), or the target's SE(3) history. SE(3)-WAM additionally uses the depth of the current frame to give each patch its metric 3D position. DP: Diffusion Policy. Best in bold. Per-family and per-task results, contact errors, and intervals in Appendix C.1.

Method	History			SR (%)			
	Wrist	RGB	Flow	SE(3)	Native	+150 ms	Paused	p95 lat. (ms)
ACT [66]					42.0	24.1	40.9	10.7
DP [10]		✓			0.0	0.0	—	1701.9
$\pi_{0.5}$ [35]					—	—	—	—
$\pi_{0.5}$+SE(3)	—			✓	—	—	—	—
DynamicVLA [54]		✓			—	—	—	—
PUMA [12]			✓		—	—	—	—
DreamVLA [63]		✓			—	—	—	—
OpenWAM-α [49]					—	—	—	—
DynamicWAM [31]			✓		—	—	—	—
DynamicWAM+SE(3)	—		✓	✓	—	—	—	—
SE(3)-WAM				✓	—	—	—	—

Table 2. **DOMINO results** on 24 single-arm clean-L1 tasks, each method trained on DOMINO's demonstrations (one training seed per method, 50 episodes per task and method). SR (%) is grouped by interaction type and averaged over tasks; MS is the manipulation score. Protocol: Appendix C.3.

Method	Intercept	Track	All	MS
$\pi_{0.5}$	—	—	—	—
DynamicWAM	—	—	—	—
PUMA	—	—	—	—
SE(3)-WAM	—	—	—	—

Figure placeholder: Fig. 5 — ablation effects by task

Layout: column width, two heatmaps side by side with shared row labels: left *native latency*, right *+150 ms*. **Rows (top to bottom):** no target history; no future supervision; no scene-flow supervision; no dedicated SE(3); uniform-fidelity forecast. **Columns:** All; the five families (Grasp, Place, Pass, Capture, Impact); a thin separator; then two task groups fixed in advance from Table S3: *rotation-sensitive* (tasks whose terms include $r\theta$) and *context-active* (tasks with an obstacle clearance m or a moving context: belt, ramp, striker, rotating wire). **Cells:** ΔSR = control minus full model in points, mean of three training seeds, printed with one decimal; diverging color centered at 0 (red = drop, blue = gain), one symmetric scale for both heatmaps, saturated beyond ± 15 points; bold value when the paired 95% bootstrap interval excludes 0. **Rules:** same held-out scenes for all rows; group membership listed in the appendix; no t3_phase exclusion here (sensitivity analysis in the appendix).

Figure 5. **Ablation effects by task.** SR difference of each control to the full model (points) with native latency (left) and 150 ms of added latency (right), by task family and by two task groups fixed from Table S3: tasks whose error depends on rotation ($r\theta$) and tasks whose context constrains or drives the target. Bold: the paired 95% interval excludes zero. [Summarize where each component matters.]

with their original executors changes their SR by at most [X.X] points (Appendix C.2).

DOMINO. We also train SE(3)-WAM, $\pi_{0.5}$, DynamicWAM, and PUMA on 24 single-arm Franka tasks of DOMINO [12] with its demonstrations and evaluate them with its native evaluator. SE(3)-WAM reaches [XX.X]% SR against [XX.X]% for [the strongest baseline] (Table 2; Appendix C.3). As on Dynamic-14, it reads the simulator pose of the target.

4.3. Ablations and Analysis

Five controls each change one component of the final model and keep the data, recipe, and training budget, with three training seeds each (Table 3, Fig. 5). They test the argument of Sec. 3.2: the target history supplies the velocity on which e_v in Eq. (2) depends, future supervision shapes the representation without adding inference time, and the two geometric components and a uniform-fidelity forecast test the allocation of precision by role. We evaluate them with native latency and with 150 ms of added latency (per-seed results and a sensitivity analysis in Appendix C.4).

Table 3. **Core ablations** of the final 24-layer model on held-out Dynamic-14 scenes (three training seeds). Each row changes one component of SE(3)-WAM (*). Uniform-fidelity forecast: the SE(3) and scene-flow targets replaced by a forecast of the whole frame at one fidelity. Native, +150 ms: success (%) with native latency and with 150 ms of added latency. p95 lat.: p95 latency, the time-to-action (ms).

Variant	Native	+150 ms	p95 lat. (ms)
SE(3)-WAM *	—	—	—
No target history	—	—	—
No future supervision	—	—	—
No scene-flow supervision	—	—	—
No dedicated SE(3)	—	—	—
Uniform-fidelity forecast	—	—	—

Table 4. **Real-robot results.** Success rate (%) on the six tasks of Sec. 4.4, 20 trials per task and method; All pools the trials. p95 lat.: p95 latency, the end-to-end time-to-action (ms) on one RTX 5090, including tracking for SE(3)-WAM. DP: Diffusion Policy. Intervals in Appendix D.

Method	T1	T2	T3	T4	T5	T6	All	p95 lat.
$\pi_{0.5}$	—	—	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—	—	—
DP	—	—	—	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—	—	—	—

Target history. The no-history control removes the past target poses with their velocity and acceleration features and keeps the current image, the current target pose, and the robot history; unlike the other controls, it changes the input. Without the history, SR drops by [X.X] points with native latency and by [X.X] points with 150 ms of added latency, as Eq. (2) predicts: a longer lead amplifies the velocity error that the history removes. Because the history relies on pose estimates, we also perturb them; under synthetic correlated pose noise, SE(3)-WAM keeps [XX.X]% SR (Appendix C.5).

Future supervision. The no-future control removes all forecast losses, the SE(3), scene-flow, and future appearance targets, and keeps the current-target RGB reconstruction. Its inference is identical to that of SE(3)-WAM, yet SR drops by [X.X] points with native latency and by [X.X] points with 150 ms of added latency, most on [tasks with non-uniform target motion]. Offline, the learned SE(3) forecast is also more accurate than constant-velocity and constant-acceleration extrapolation of the same history (Table S11).

Allocation of precision. Three controls keep all auxiliary targets and change the geometric forecast. Removing the scene-flow supervision lowers SR by [X.X] points; because

Figure placeholder: Fig. 6 — real-robot tasks and a paired trial

Layout: full width, two rows of six equal panels with 16:9 crops, so that both rows share one panel size. **(a) Top row, six equal panels, one per task (T1–T6):** the real photo at the moment SE(3)-WAM grasps or intercepts the target; overlays: 3–4 semi-transparent past poses of the target from the tracker and an arrow for its motion (blue), drawn as in Fig. 4; task name and motion type under each panel (e.g., "T5 post-impact interception: deflection after the collision"). **(b) Bottom row, a paired trial on one task ([T5 or T6]; not the linear-motion grasp of Fig. 1):** left three panels SE(3)-WAM, right three panels the strongest baseline, on the same trial key; in each half, the query frame at τ_t, the frame at the execution time τ^{a} of the first executed command, and contact; overlays: blue tracked past poses and purple commanded gripper path with a marker where the gripper closes; in the baseline row, a red segment from the closing point to the target with its length in mm. Under each half, a timeline strip marking capture, first execution, and contact, annotated with the method's p95 latency from Table 4; outcome and criterion above each half. **Rules:** real photos, unretouched; overlays from the logged data of the shown trials; same crop and scale within (b).

Figure 6. **Real-robot experiments.** (a) SE(3)-WAM on the six tasks at contact. (b) SE(3)-WAM and [the strongest baseline] on one trial key of [task]: [e.g., the baseline acts XX ms later and closes XX mm behind the target].

the flow also covers the target and the robot, and the appearance targets remain, this measures the contribution of coarse scene-motion supervision rather than of all context information. Removing the dedicated SE(3) supervision leaves the target with only the coarse flow that every part of the scene receives and lowers SR by [X.X] points. If precision follows the roles, the SE(3) supervision should matter most on tasks with a tight pose tolerance or a rotation-sensitive contact, and the flow supervision on tasks whose context constrains or drives the target. [Whether Fig. 5 shows this pattern; if it does not, state that the controls establish the contribution of each component but not the allocation by role.] The uniform-fidelity control replaces the SE(3) and scene-flow targets by [future latents of a frozen video VAE] of the whole frame at the same horizons, so that every part of the scene is forecast at one fidelity; at the same inference cost, it trails SE(3)-WAM by [X.X] points. [One-sentence summary of which components matter, and where.]

4.4. Real-Robot Experiments

Setup. The real robot tests the complete system, with estimated poses and physical latency. We evaluate six tasks on a Franka arm ([model]) with [gripper model] and a fixed RealSense D435 RGB-D camera (Fig. 6a; Appendix D) that repeat the motion types of Dynamic-14: conveyor grasping and placement (T1, T2; grasp and place families), turntable grasping (T3; rotating target), grasping a moving toy car (T4; free planar motion), ball interception after a collision (T5; impact), and interception of objects rolling or sliding down a ramp (T6; capture). We compare SE(3)-WAM with one baseline per class: $\pi_{0.5}$ (VLA), DynamicWAM (dynamic WAM), and Diffusion Policy [10] (reactive). Each method is trained or fine-tuned per task on the same 100 teleoperated demonstrations and runs 20 trials per task with shared trial keys (Appendix D.3). The baselines receive RGB [and, if their recipe uses one, a wrist camera], whereas SE(3)-WAM also receives the target pose from FoundationPose [50], updated at every control-period boundary before its query (Appendix D.2). All latencies in Table 4 are end-to-end.

Results. SE(3)-WAM succeeds in [XX.X]% of all trials, against [XX.X]% for [the strongest baseline], a paired difference of [X.X] points (95% interval [[X.X, X.X]]; Table 4). Its p95 latency, including tracking, is [XX] ms, [within one control period], whereas [name] needs [XX] ms. The margin is largest on [tasks] (Fig. 6b), and the dominant failure of SE(3)-WAM is [failure mode] (Appendix D.4).

5. Conclusion

SE(3)-WAM spends precision where contact depends on it: a metric SE(3) history and forecast of the target and a sparse 3D flow of the whole scene, with the forecasts used only as training supervision. It reaches [XX.X]% success on Dynamic-14, [XX.X]% on DOMINO, and [XX]% on six real-robot tasks at a p95 latency of [XX] ms, improving on reactive, dynamic, and world–action policies. [The ablations show that the target history matters most under added latency and that future supervision improves control without changing the inference computation.] [State what the dedicated SE(3), scene-flow, and uniform-fidelity controls support: a better forecast design, or, if the task groups also agree, precision allocated by role.] Accounting for inference latency [state how success rates and relative performance differ from paused evaluation].

Limitations. SE(3)-WAM requires a specified rigid target and reliable estimates of its pose, and its forecast does not model how the robot's own contact changes the target's motion. Its target-trajectory head predicts a single future per horizon and does not represent uncertainty or multimodal motion, which an uncertainty-aware forecast could address.

LLM Usage Statement

[To be completed before submission: actual tools used, affected parts of the research and manuscript, and extent of use.]

References

[1] Peter K. Allen, Aleksandar Timcenko, Billibon Yoshimi, and Paul Michelman. Automated tracking and grasping of a moving object with a robotic hand-eye system. *IEEE Transactions on Robotics and Automation*, 9(2):152–165, 1993. 2

[2] Yaakov Bar-Shalom, X. Rong Li, and Thiagalingam Kirubarajan. *Estimation with Applications to Tracking and Navigation*. Wiley, 2001.

[3] Berthold Bäuml, Thomas Wimböck, and Gerd Hirzinger. Kinematically optimal catching a flying ball with a hand-arm-system. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2010. 2

[4] Homanga Bharadhwaj, Roozbeh Mottaghi, Abhinav Gupta, and Shubham Tulsiani. Track2Act: Predicting point tracks from internet videos enables generalizable robot manipulation. In *European Conference on Computer Vision*, 2024. 2

[5] Kevin Black, Noah Brown, Danny Driess, et al. π_0: A vision-language-action flow model for general robot control. In *Proceedings of Robotics: Science and Systems*, 2025. 1

[6] Kevin Black, Manuel Y. Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. In *Advances in Neural Information Processing Systems*, 2025. 2, 5

[7] Arunkumar Byravan and Dieter Fox. SE3-Nets: Learning rigid body motion using deep neural networks. In *IEEE International Conference on Robotics and Automation*, 2017. 2

[8] Arunkumar Byravan, Felix Leeb, Franziska Meier, and Dieter Fox. SE3-Pose-Nets: Structured deep dynamics models for visuomotor control. In *IEEE International Conference on Robotics and Automation*, 2018. 2

[9] Chongkei Chang and Zhidong Deng. DynaWM: A base-VLA-guided world foundation model for moving-object manipulation. *arXiv preprint arXiv:2607.02604*, 2026. 2

[10] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin C. M. Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems*, 2023. 1, 6, 8, 18, 22

[11] David Emukpere, Romain Deffayet, Bingbing Wu, Romain Brégier, Michael Niemaz, Jean-Luc Meunier, Denys Proux, Jean-Michel Renders, and Seungsu Kim. Disentangled object-centric image representation for robotic manipulation. *arXiv preprint arXiv:2503.11565*, 2025. 3

[12] Heng Fang, Shangru Li, Shuhan Wang, Xuanyang Xi, Dingkang Liang, and Xiang Bai. Towards generalizable robotic manipulation in dynamic environments. *arXiv preprint arXiv:2603.15620*, 2026. 1, 2, 6, 7, 18, 20

[13] Gunnar Farnebäck. Two-frame motion estimation based on polynomial expansion. In *Proceedings of the Scandinavian Conference on Image Analysis*, pages 363–370, 2003. 23

[14] Ankit Goyal, Jie Xu, Yijie Guo, Valts Blukis, Yu-Wei Chao, and Dieter Fox. RVT: Robotic view transformer for 3D object manipulation. In *Conference on Robot Learning*, pages 694–710, 2023. 2

[15] Jun Guo, Xiaojian Ma, Yikai Wang, Min Yang, Huaping Liu, and Qing Li. FlowDreamer: A RGB-D world model with flow-based motion representations for robot manipulation. *arXiv preprint arXiv:2505.10075*, 2025. 2

[16] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16000–16009, 2022. 3, 12, 23

[17] Cheng-Chun Hsu, Bowen Wen, Jie Xu, Yashraj Narang, Xiaolong Wang, Yuke Zhu, Joydeep Biswas, and Stan Birchfield. SPOT: SE(3) pose trajectory diffusion for object-centric manipulation. In *IEEE International Conference on Robotics and Automation*, 2025. 2

[18] Yucheng Hu, Yanjiang Guo, Pengchao Wang, Xiaoyu Chen, Yen-Jen Wang, Jianke Zhang, Koushil Sreenath, Chaochao Lu, and Jianyu Chen. Video prediction policy: A generalist robot policy with predictive visual representations. In *International Conference on Machine Learning*, 2025. 2

[19] Wenlong Huang, Yu-Wei Chao, Arsalan Mousavian, Ming-Yu Liu, Dieter Fox, Kaichun Mo, and Li Fei-Fei. PointWorld: Scaling 3D world models for in-the-wild robotic manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026. 2, 13

[20] Tsung-Wei Ke, Nikolaos Gkanatsios, and Katerina Fragkiadaki. 3D diffuser actor: Policy diffusion with 3D scene representations. In *Conference on Robot Learning*, 2024. 2

[21] Alexander Khazatsky, Karl Pertsch, Suraj Nair, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *Proceedings of Robotics: Science and Systems*, 2024. 6, 13

[22] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning*, 2024. 1

[23] Fenghao Lei, Zhixiong Huang, Long Yang, Jiabao Chen, Peilin Huang, Han Fu, Zhuo Li, and Xiaoxue Ren. DELE-w0.5: Inferring action from future latent state for robotic manipulation. *arXiv preprint arXiv:2608.22067*, 2026. 2

[24] Jiajun Li, Tiecheng Guo, Yifan Ye, Rongyu Zhang, Xiaowei Chi, Qianpu Sun, Ying Li, Yunfan Lou, Jinqiao Wang, Zhihe Lu, Meng Guo, and Shanghang Zhang. Efficient-WAM: A 1B-parameter world-action model with low-cost future imagination. *arXiv preprint arXiv:2606.10040*, 2026. 2

[25] Shuang Li, Yihuai Gao, Dorsa Sadigh, and Shuran Song. Unified video action model. In *Proceedings of Robotics: Science and Systems*, 2025. 2, 5

[26] Haoran Liao, Pengyue Wang, Shuoyu Chen, Kehan Cheng, Xuhang Chen, Yuhao Lin, Mu Lin, Zhizhao Liang, Xiaoyi Fan, Chengyi Xing, Dan Niu, Yi-Lin Wei, and Wei-Shi Zheng. DynamicManip: Enabling dynamic manipulation from a single static demonstration. *arXiv preprint arXiv:2608.01452*, 2026. 2

[27] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative

modeling. In *International Conference on Learning Representations*, 2023. 3

[28] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, and Lei Zhang. Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. In *European Conference on Computer Vision*, 2024. 21

[29] Yuejiang Liu, Jubayer Ibn Hamid, Annie Xie, Yoonho Lee, Maximilian Du, and Chelsea Finn. Bidirectional decoding: Improving action chunking via guided test-time sampling. In *International Conference on Learning Representations*, 2025. 2

[30] Yushan Liu, Peibo Sun, Shoujie Li, Yifan Xie, Lingfeng Zhang, Xintao Chao, Shiyuan Dong, Fang Chen, Xiao-Ping Zhang, and Wenbo Ding. OA-WAM: Object-addressable world action model for robust robot manipulation. *arXiv preprint arXiv:2605.06481*, 2026. 2

[31] Yunfan Lou, Hewen Gao, Xiyu Zhu, Zhuoran Qiao, Xuan Han, Yifan Yang, Yifan Ye, Boxian Yao, and Zhibo Pang. DynamicWAM: Dual-path motion conditioning for world-action models in dynamic manipulation. *arXiv preprint arXiv:2608.00793*, 2026. 1, 2, 6, 18, 23

[32] Siyuan Ma, Yutian Zhang, Boshi Zhang, Qinglian Wu, Jiaqi Zhai, Dong Wei, and Xiaojin Huang. ForeTime-VLA: Causal future-token distillation from a world action model for conveyor-belt manipulation. *arXiv preprint arXiv:2608.20735*, 2026. 2

[33] Lucas Manuelli, Wei Gao, Peter Florence, and Russ Tedrake. kPAM: Keypoint affordances for category-level robotic manipulation. In *International Symposium on Robotics Research*, 2019. 2

[34] Naresh Marturi, Marek Kopicki, Alireza Rastegarpanah, et al. Dynamic grasp and trajectory planning for moving objects. *Autonomous Robots*, 43(5):1241–1256, 2019. 2

[35] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. 1, 6, 18

[36] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment anything in images and videos. In *International Conference on Learning Representations*, 2025. 21

[37] Yiran Shi, Dongqi Guo, Tianchen Zhao, Feng Gao, Liangzhi Shi, Chao Yu, ZhiJian Mo, Qihua Xiao, XiaoShuai Peng, Qingmin Liao, and Yu Wang. StreamingVLA: Streaming vision-language-action model with action flow matching and adaptive early observation. *arXiv preprint arXiv:2603.28565*, 2026. 2

[38] Mohit Shridhar, Lucas Manuelli, and Dieter Fox. Perceiver-actor: A multi-task transformer for robotic manipulation. In *Conference on Robot Learning*, 2022. 2

[39] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, et al. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025. 2

[40] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, et al. DINOv3. *arXiv preprint arXiv:2508.10104*, 2025. 5, 12, 13, 23

[41] Otto J. M. Smith. Closer control of loops with dead time. *Chemical Engineering Progress*, 53(5):217–219, 1957. 2

[42] Rustin Soraki, Homanga Bharadhwaj, Ali Farhadi, and Roozbeh Mottaghi. ObjectForesight: Predicting future 3D object trajectories from human videos. *arXiv preprint arXiv:2601.05237*, 2026. 2

[43] Yue Su, Sijin Chen, Haixin Shi, Mingyu Liu, Zhengshen Zhang, Ningyuan Huang, Weiheng Zhong, Zhengbang Zhu, Yuxiao Liu, and Xihui Liu. World guidance: World modeling in condition space for action generation. *arXiv preprint arXiv:2602.22010*, 2026. 2

[44] Shahram Najam Syed, Arthur Jakobsson, Haoran Hao, and Jeffrey Ichnowski. Intercepting the future: Latent-space predictive world model for dynamic VLA manipulation. *arXiv preprint arXiv:2606.02486*, 2026. 1, 2

[45] Jiaming Tang, Yufei Sun, Yilong Zhao, Shang Yang, Yujun Lin, Zhuoyang Zhang, James Hou, Yao Lu, Zhijian Liu, and Song Han. VLASH: Real-time VLAs via future-state-aware asynchronous inference. *arXiv preprint arXiv:2512.01031*, 2025. 2

[46] Yang Tian, Sizhe Yang, Jia Zeng, Ping Wang, Dahua Lin, Hao Dong, and Jiangmiao Pang. Predictive inverse dynamics models are scalable learners for robotic manipulation. In *International Conference on Learning Representations*, 2025. 2

[47] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. 5, 14

[48] Changhao Wang, Jaden Clark, and Shuran Song. Chronos policy: Learning robot policies on physical time. *arXiv preprint arXiv:2610.13929*, 2026. 2

[49] Yuran Wang, Siqiao Huang, Mingleyang Li, Chenhao Zhang, Jiaqi Liang, Weiyang Jin, Yue Chen, Xuemin Chi, Donghao Zhou, Qize Yu, Yu-Kai Wang, Yuhan Rui, Shenzhe Yao, Zhen Yuan, Zhenhao Shen, Kefei Zhu, Zijie Zhu, Ning Gao, Xiaowei Chi, Guanqi He, Shanghang Zhang, Hao Dong, Lin Shao, and Hang Zhao. OpenWAM: An open, modular exploration towards systematic world-action model pretraining. *arXiv preprint arXiv:2609.07398*, 2026. 1, 2, 6, 18

[50] Bowen Wen, Wei Yang, Jan Kautz, and Stan Birchfield. FoundationPose: Unified 6D pose estimation and tracking of novel objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17868–17879, 2024. 8, 21

[51] Chuan Wen, Xingyu Lin, John So, Kai Chen, Qi Dou, Yang Gao, and Pieter Abbeel. Any-point trajectory modeling for policy learning. In *Proceedings of Robotics: Science and Systems*, 2024. 2

[52] Gaoyuan Wu, Ziyu Shan, Haoyang Du, Yuyao Jiang, and Ziwei Wang. AffordTrajDP: Dynamic affordance-guided

visuomotor policy learning for robotic manipulation. *arXiv preprint arXiv:2608.01603*, 2026. 2

[53] Hongtao Wu, Ya Jing, Chilam Cheang, Guangzeng Chen, Jiafeng Xu, Xinghang Li, Minghuan Liu, Hang Li, and Tao Kong. Unleashing large-scale video generative pre-training for visual robot manipulation. In *International Conference on Learning Representations*, 2024. 2

[54] Haozhe Xie, Beichen Wen, Jiarui Zheng, Zhaoxi Chen, Fangzhou Hong, Haiwen Diao, and Ziwei Liu. DynamicVLA: A vision-language-action model for dynamic object manipulation. *arXiv preprint arXiv:2601.22153*, 2026. 2, 6, 18

[55] Mengda Xu, Zhenjia Xu, Yinghao Xu, Cheng Chi, Gordon Wetzstein, Manuela Veloso, and Shuran Song. Flow as the cross-domain manipulation interface. In *Conference on Robot Learning*, 2024. 2

[56] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026. 1, 2

[57] Hanyang Yu, Haitao Lin, Jingbo Zhang, Wenyao Zhang, Chenghao Gu, Heng Li, and Ping Tan. MaskWAM: Unifying mask prompting and prediction for world-action models. *arXiv preprint arXiv:2606.13515*, 2026. 2

[58] Chengbo Yuan, Chuan Wen, Tong Zhang, and Yang Gao. General flow as foundation affordance for scalable robot learning. In *Conference on Robot Learning*, pages 1541–1566, 2024. 2

[59] Shanglin Yuan, Weiheng Zhao, Xin Shi, Haoyi Jiang, Xianda Guo, Liu Liu, Wenyu Liu, Wei Sui, and Xinggang Wang. DreamWAM: Beyond RGB future prediction for world action models. *arXiv preprint arXiv:2608.04996*, 2026. 2

[60] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-WAM: Do world action models need test-time future imagination? *arXiv preprint arXiv:2603.16666*, 2026. 2, 5

[61] Peng Yun, Shouwang Huang, Hao Li, Jinxi Li, Jianan Wang, and Bo Yang. PhysMani: Physics-principled 3D world model for dynamic object manipulation. *arXiv preprint arXiv:2607.01938*, 2026. 2

[62] Yanjie Ze, Gu Zhang, Kangning Zhang, Chenyuan Hu, Muhan Wang, and Huazhe Xu. 3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations. In *Proceedings of Robotics: Science and Systems*, 2024. 2

[63] Wenyao Zhang, Hongsi Liu, Zekun Qi, Yunnan Wang, XinQiang Yu, Jiazhao Zhang, Runpei Dong, Jiawei He, He Wang, Zhizheng Zhang, Li Yi, Wenjun Zeng, and Xin Jin. DreamVLA: A vision-language-action model dreamed with comprehensive world knowledge. In *Advances in Neural Information Processing Systems*, pages 24195–24228, 2025. 1, 2, 6, 18

[64] Yifan Zhang, Ruiping Wang, and Xilin Chen. Dynamic behavior cloning with temporal feature prediction: Enhancing robotic arm manipulation in moving object tasks. *IEEE Robotics and Automation Letters*, 10(6):5209–5216, 2025. 2

[65] Ruiteng Zhao, Zhengshen Zhang, Yue Su, Wenshuo Wang, Jiahui Li, Zhiyuan Yang, Francis E. H. Tay, Marcelo H. Ang, and Haiyue Zhu. SG-WAM: Self-guided world modeling in geometry-aware policy space. *arXiv preprint arXiv:2608.01397*, 2026. 2

[66] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems*, 2023. 6, 18

[67] Ruijie Zheng, Yongyuan Liang, Shuaiyi Huang, Jianfeng Gao, Hal Daumé III, Andrey Kolobov, Furong Huang, and Jianwei Yang. TraceVLA: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies. In *International Conference on Learning Representations*, 2025. 2

[68] Hongyan Zhi, Peihao Chen, Siyuan Zhou, Yubo Dong, Quanxi Wu, Lei Han, and Mingkui Tan. 3DFlowAction: Learning cross-embodiment manipulation from 3D flow world model. *arXiv preprint arXiv:2506.06199*, 2025. 2

[69] Chuning Zhu, Raymond Yu, Siyuan Feng, Benjamin Burchfiel, Paarth Shah, and Abhishek Gupta. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets. In *Proceedings of Robotics: Science and Systems*, 2025. 2

[70] Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Kevin Lin, Abhiram Maddukuri, Soroush Nasiriany, and Yifeng Zhu. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020. 5, 14

[71] Brianna Zitkovich, Tianhe Yu, Sichun Xu, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, 2023. 1

A. Method and Training Details

A.1. Error Decomposition

This subsection derives the decomposition of Eq. 2 that motivates the design of SE(3)-WAM (Sec. 3.2). It describes how the errors of an estimate scale with the lead and the motion; it is not a model of success.

Translation. Let the true target position at the execution time be $\mathbf{p}(\tau^{\mathrm{a}}) = \mathbf{p}(\tau_t) + \mathbf{v}\Delta + \boldsymbol{\rho}$, where $\mathbf{v}$ is the velocity at τ_t and $\boldsymbol{\rho}$ the remaining motion, and let the policy's estimate be $\hat{\mathbf{p}}(\tau^{\mathrm{a}}) = \hat{\mathbf{p}}(\tau_t) + \hat{\mathbf{v}}\hat{\Delta} + \hat{\boldsymbol{\rho}}$. Subtracting and adding $\mathbf{v}\hat{\Delta}$ gives the identity

$$\begin{aligned}\hat{\mathbf{p}}(\tau^{\mathrm{a}}) - \mathbf{p}(\tau^{\mathrm{a}}) = \big[\hat{\mathbf{p}}(\tau_t) - \mathbf{p}(\tau_t)\big] + (\hat{\mathbf{v}} - \mathbf{v})\hat{\Delta} \\ + \mathbf{v}(\hat{\Delta} - \Delta) + (\hat{\boldsymbol{\rho}} - \boldsymbol{\rho}),\end{aligned} \tag{S1}$$

and the triangle inequality yields Eq. 2 with $e_v = \|\hat{\mathbf{v}} - \mathbf{v}\|$ and $e_{\mathrm{dyn}} = \|\hat{\boldsymbol{\rho}} - \boldsymbol{\rho}\|$. The residual $\boldsymbol{\rho}$ is exactly the part of the motion that constant-velocity extrapolation misses: under a constant acceleration $\mathbf{a}$, $\boldsymbol{\rho} = \frac{1}{2}\mathbf{a}\Delta^2$, so an acceleration estimate from the history removes part of e_{dyn}, whereas impacts, rolling contact, and friction changes leave a remainder that only a model of the dynamics can reduce. For a learned predictor, the split of its estimate into $\hat{\mathbf{v}}\hat{\Delta}$ and $\hat{\boldsymbol{\rho}}$ is not unique, so the decomposition states which information each term requires rather than how a particular network uses it.

Rotation and the contact point. With $R(\tau^{\mathrm{a}}) = \mathrm{Exp}(\boldsymbol{\omega}\Delta + \boldsymbol{\eta})R(\tau_t)$ and $\hat{R}(\tau^{\mathrm{a}}) = \mathrm{Exp}(\hat{\boldsymbol{\omega}}\hat{\Delta} + \hat{\boldsymbol{\eta}})\hat{R}(\tau_t)$, rotations about a common fixed axis compose additively, and

$$\theta = \big\| \mathrm{Log}(\hat{R}(\tau^{\mathrm{a}})R(\tau^{\mathrm{a}})^{\top})^{\vee} \big\| \le \theta_{\mathrm{loc}} + \hat{\Delta}\, e_\omega + \|\boldsymbol{\omega}\|\, e_\tau + e_\eta, \tag{S2}$$

with the current orientation error θ_{loc} and e_ω and e_η defined as for translation; for general rotations, the Baker–Campbell–Hausdorff formula adds terms of second order in the angles. A contact point at $\mathbf{c}$ in the object frame, with $\|\mathbf{c}\| = r$, has the world position $\mathbf{p} + R\mathbf{c}$, so its error is $(\hat{\mathbf{p}} - \mathbf{p}) + (\hat{R}R^{\top} - I)R\mathbf{c}$. Since the operator norm of $\hat{R}R^{\top} - I$ for a rotation by θ is $2\sin(\theta/2)$,

$$e_{\mathrm{c}} \le \|\hat{\mathbf{p}} - \mathbf{p}\| + 2r\sin(\theta/2) \le \|\hat{\mathbf{p}} - \mathbf{p}\| + r\theta. \tag{S3}$$

Orientation errors therefore enter the contact scaled by the lever arm, and a grasp or placement that depends on orientation needs the full SE(3) state of the target.

Obstacles. Suppose that the executed path keeps every point of the robot and the carried object at least m away from the *predicted* positions of the context points it passes. A collision at a context point then requires the true position of that point to differ from its prediction by at least m. Under this assumption, context errors below the clearance do not cause collisions, which is why a coarse estimate of the surroundings can suffice when $m \gg \varepsilon$. The assumption concerns the executed path: SE(3)-WAM does not enforce a margin explicitly, so the effect of coarse context supervision is tested empirically by the ablations (Sec. 4.3). A flow sampled on a grid can also miss structures thinner than its spacing.

Drivers. A driver, such as a conveyor, a ramp, or a striker, matters through the motion it imparts to the target. Its effect on the contact therefore enters Eq. 2 through the target's own terms, mostly through e_{dyn} when the driver changes the target's motion, as at an impact. A policy needs to know the driver's motion only well enough to anticipate when and how it will act on the target; the resulting motion of the target is what the dedicated SE(3) forecast supervises.

Scope. The decomposition addresses the approach to a rigid target whose motion is exogenous until contact. It does not cover deformable or articulated targets, whose motion is not determined by six numbers, or the motion after contact, which depends on the action itself and would call for action-conditioned prediction. Scenes in which the role of a body changes over time, e.g., sequential multi-object tasks, are also outside its scope.

A.2. Input Tokens

Current image. A frozen MAE ViT-B/16 [16] encodes the 224×224 agent-view RGB into 196 patch tokens; we chose it over frozen DINOv3 encoders [40] and a CNN trained with the policy on development scenes (Appendix E.1). Each patch receives a view embedding, a linear embedding of its mean valid metric depth, and a linear embedding of the base-frame position of its back-projected center pixel, computed with the camera intrinsics and extrinsics. These positions also enter 3D rotary position embeddings of the patch queries and keys, and the same center points are the source points of the sparse 3D flow (Appendix A.3). Target, robot, summary, and action tokens receive no rotary embedding. The 196 patch tokens, 8 target-state tokens, and 8 robot tokens form the 212 scene tokens; past images never enter the model.

Target-motion history. The target history covers the last 8 control steps, at the offsets $-350, -300, \ldots, 0$ ms. Each of the 8 target-state tokens is a 25-dimensional descriptor $(\mathbf{p}, \log R, \Delta\mathbf{p}, \log \Delta R, \Delta t, \mathbf{v}, \boldsymbol{\omega}, \mathbf{a}, \boldsymbol{\alpha})$ with dimensions $(3, 3, 3, 3, 1, 3, 3, 3)$. Slots 0–6 describe the seven intervals between consecutive offsets, and slot 7 holds the current absolute pose in the base frame. Velocities divide interval displacements by the elapsed time, and accelerations divide adjacent velocity differences by the spacing of the

interval centers. We standardize each dimension with training statistics, zero invalid dimensions, and append three validity flags (pose, interval, and acceleration) before a $28 \to 1024 \to 1024$ GELU MLP with an additive relative-time embedding. A slot without any valid feature is masked out of attention. On the real robot, we resample the tracker's estimates (Appendix D.2) at the same offsets, linearly in translation and along the geodesic in rotation, without extrapolation and without interpolating across gaps longer than two camera periods.

Robot state and summary tokens. 8 end-effector states (position, rotation vector, and measured gripper opening fraction) at the same offsets form the robot tokens. 16 learned embeddings form the summary tokens. The policy has no language token, and the view, time, depth, and position embeddings modify existing tokens rather than adding slots.

A.3. Forecast Heads and Losses

All forecast heads are light readouts of the final hidden states of the world stream. They are trained jointly with the action stream and skipped at deployment, and no token reads their outputs.

Target. A linear map from the mean final state of the summary tokens gives $\{(\hat{\mathbf{d}}_h, \hat{\boldsymbol{\phi}}_h)\}_{h=1}^{8}$, trained with $\mathcal{L}_P$ of Eq. 4 and $\lambda_R = 0.1$.

Sparse 3D flow. The flow is the cumulative base-frame displacement of the 196 patch center points at the 8 horizons, together with their visibility. A linear readout on each patch token gives its point's displacement and visibility logit at every horizon, trained with $\mathcal{L}_F$ of Eq. 5 and the scale $s = 0.7154605$ m. The visibility loss has weight 1 in simulation, and points that are valid but invisible still supervise their displacement. The points cover the target, the robot, and the surroundings alike.

Auxiliary targets. Linear readouts of the final patch tokens predict three future appearance targets at every horizon: metric depth and moving-surface RGB through 16×16 pixel readouts per patch, and the 1024-dimensional features of a frozen DINOv3 ViT-L/16 teacher [40] on the 14×14 grid. Depth uses squared error on valid pixels, the DINO features squared error after channel-wise layer normalization, and RGB squared error on surfaces that move by more than 1 mm, including the robot. A $1024 \to 256 \to 3072$ MLP with GELU and a final sigmoid reads the mean of the final summary tokens and reconstructs the current target's 32×32 RGB region with an absolute error. The four losses form $\mathcal{L}_{\text{aux}}$, each with weight $\lambda_{\text{aux}} = 0.1$.

A.4. Action Stream and Backbone

Commands. Each chunk has 8 seven-dimensional commands at a control period of 50 ms, each with a base-frame translation increment, a left-relative rotation vector, and a gripper command. Translation and nonzero rotation increments refer to the measured end-effector pose at application. Command j of the chunk for query t is labeled with the demonstration command issued at $\tau_t + j\Delta_a$. The task's command delay d_{task} is added on application (Eq. 1); it does not shift the training indices.

Flow matching. Normalization uses the mean and population standard deviation of the valid issued commands of the training split. Flow matching uses a unit Gaussian in these coordinates with one flow time u per chunk, which the action tokens receive through adaptive layer normalization. Each prediction position draws 8 independent noise–time pairs and averages their losses, which reduces the variance of the action gradient without enlarging the batch. At deployment, we integrate 5 Euler steps, reverse the normalization, decode the rotation vectors, and clip each axis to 0.05 m and 0.5 rad; the gripper follows the sign of its command.

Backbone and mask. Each of the 24 layers applies one attention over all tokens with shared query, key, value, and output projections, followed by separate SiLU feed-forward networks (ratio 4) for the world and the action tokens, for 504.0M backbone parameters. The mask acts on the scene ($\mathcal{S}$), the summary tokens ($\mathcal{M}$), and the actions ($\mathcal{A}$): scene tokens attend to $\mathcal{S}$, summary tokens to $\mathcal{S}$ and $\mathcal{M}$, and action tokens to $\mathcal{M}$ and $\mathcal{A}$. A blocked edge has the logit $-\infty$ in every layer and head, in training and at deployment. Because the world tokens attend neither to the actions nor to the flow time, their states are computed once per query and reused across the Euler steps. Appendix B.3 specifies how the resulting chunks are executed.

A.5. Training on Dynamic-14

Table S1 lists the recipe on Dynamic-14. Each control step of a demonstration anchors one training window, which gives 1,324,374 windows from the 11,200 demonstrations; 350 further episodes form the validation split. A window holds 8 query positions, each with its own causal history and no attention across positions. Each loss is normalized by its count of valid labels over the accumulated global batch. The DINOv3 features of future frames are cached once per episode.

A.6. Pretraining on DROID and Fine-Tuning

Data. The real-robot base trains on DROID [21], recorded at 15 Hz, with the 3D point tracks and optimized camera calibrations that PointWorld released for it [19]. We associate each released clip with its raw DROID episode by exact

Table S1. Training hyperparameters on Dynamic-14.

Item	Value
Backbone	24 layers, width 1024, 16 heads, FFN 4096, 504.0M parameters
Tokens	212 scene, 16 summary, 8 action
Encoder	frozen MAE ViT-B/16, 224×224 input
Optimizer	AdamW, $\beta = (0.9, 0.999)$, $\epsilon = 10^{-8}$, weight decay 10^{-4}
Learning rate	2×10^{-4}, 5,174 warm-up updates from 0, cosine decay to 0
Batch, updates	global batch 512 windows, 20 epochs (51,740 updates)
Loss weights	$\lambda_P = \lambda_F = 1$, $\lambda_R = 0.1$, $\lambda_{\text{aux}} = 0.1$ per target
Precision	trainable weights FP32, frozen encoder BF16, gradient clipping at 0.1
Action noise	8 noise–time pairs per query position
Sampling	5 Euler steps for the action
Inference	FP32, world states cached within a query
Hardware	[N] RTX PRO 6000 GPUs, [X] hours per run

agreement of the joint sequence, end-effector poses, gripper states, commands, and camera identity, and exclude clips with a non-monotonic capture clock or an ambiguous association. The model sees the first exterior camera with its depth and PointWorld's calibration. Of 42,935 associated episodes, 42,087 yield 598,889 anchor windows, 538,740 from 37,874 training episodes and 60,149 from 4,213 validation episodes, with no shared episode. Each anchor expands into 8 consecutive query positions with their own causal histories; the anchor carries the flow and DINOv3 labels, and the other positions carry action labels only. All times are capture timestamps: the robot history uses the 8 most recent states at or before the query time, and the flow and DINOv3 targets keep PointWorld's record times rather than being interpolated to 50 ms horizons. Robot states and commands are expressed in the base frame after the calibrated flange-to-TCP transform. We keep the 96.88% of action slots at which the collector issued a command, without a filter on magnitude or speed, so that the base is not biased toward slow motion.

What the base learns. DROID provides no object poses, so the base has no target-state tokens and no pose head. Its inputs are the current RGB-D frame and the robot history (204 scene tokens), with unavailable depth masked, and its objectives are the action loss (weight 1), the sparse 3D flow (weight 1), and the future DINOv3 features (weight 0.1). PointWorld's tracks record only visible positions, so the flow labels contain no negative visibility examples, and the visibility loss is off. Most objects in DROID move only when the robot moves them, so the base mainly learns metric geometry, the motion caused by the robot's own actions, and its coupling with the actions; the exogenous motion of our tasks and the target pose enter during fine-tuning.

Flow labels. Each patch of the 14×14 grid follows one PointWorld track over the window. Among the tracks that start inside the patch, are initially visible, and have a valid positive depth, we prefer the one with the largest image motion whose 3D displacement agrees with its image motion, and otherwise take the track nearest to the patch center. This rule covers moving objects that a fixed center point would miss: on 18 segments with visible object motion, it placed a dynamic track inside the object in 13, against 5 for the center rule. The label is the track's displacement from its initial position, and patches without a valid track are invalid; 47.05% of the point–horizon slots carry a label.

Recipe. The base uses 48 layers at width 1024 (1.02B trainable parameters) and trains for 20 epochs (21,060 updates, 2,106 warm-up updates) at a global batch size of 512 on 64 GPUs, with the optimizer and precision of Table S1.

Fine-tuning. Each real-robot policy is initialized from the base and fine-tuned on the demonstrations of its task. The target-state tokens, the pose head, and the appearance heads that the base lacks start from fresh initialization, and the policy receives poses from the tracker of Appendix D.2. No real-robot policy is initialized from simulation weights, and the base sees no simulation data.

B. Dynamic-14 and Evaluation Protocol

B.1. Tasks, Roles, and Success Conditions

We built Dynamic-14 on MuJoCo and robosuite [47, 70]. It contains fourteen tasks in five families (Fig. 4). Table S2 lists their success conditions and Table S4 their scenario ranges. We will release the tasks, the accepted scenarios, the success checks, and the demonstrations.

Roles. Table S3 assigns the bodies of each task to the target and context roles of Sec. 3.1. A moving receptacle is therefore a target, since its pose decides containment. The wire of the passage tasks plays both roles: its pose at the entry is a target, because the wand must be aligned within 30 mm there, and its geometry along the loop is context, because any contact fails. Conveyors, turntables, ramps, and strikers are drivers in the context. The last column lists the error terms of Eq. 2 that each task exercises (Appendix A.1). The two task groups of Fig. 5 follow from this table and were fixed before the ablations: the rotation-sensitive group contains the tasks whose terms include $r\theta$ (`grasp`, `t1_so3`, `t2_se3`, `t3_phase`, `t3_full`, `t4_can`, `t5_replan`), and the

Table S2. Success conditions of the fourteen Dynamic-14 tasks, with the thresholds of our task implementations. A task must pass its preceding stages and avoid terminal failures. All tasks run at 20 Hz with a 2 ms physics step; durations below refer to physical time. COM denotes center of mass.

Task	Ordered success conditions	Limit
grasp; t1_belt, t1_xyz, t1_so3, t1_occ	Approach within 0.10 m; bilateral fingerpad contact for 0.05 s; collision-geometry bottom at least 0.05 m above support; retain contact and clearance for 0.50 s. T1 adds conveyor drive, translation, rotation, or a physical occluder. SO3 has no additional success-angle threshold; unsafe occluder contact terminates OCC.	160 steps 8 s
t2_trans	Approach within 0.10 m. Entire object collision geometry inside the receptacle: planar clearance and floor gap both ≥ -0.002 m, top no higher than the opening. Release requires an opening command < -0.5 and no fingerpad contact. Relative linear/angular speeds ≤ 0.08 m/s and 0.8 rad/s for 0.10 s, followed by a 0.50 s stable dwell.	200 10 s
t2_se3	Approach within 0.05 m and 0.12 rad. Full T-shaped containment: planar clearance ≥ 0.002 m, floor gap in $[-0.002, 0.002]$ m, tilt ≤ 0.05 rad, top below opening. After release, relative speeds ≤ 0.02 m/s and 0.04 rad/s for 0.10 s, then a 0.50 s dwell. The 0.12 rad gate applies to approach.	200 10 s
t3_phase	The episode starts with the wand already grasped. Phase alignment: distance ≤ 0.090 m and loop-direction error ≤ 0.20 rad. Enter within 0.030 m at progress ≤ 0.15; reach historical maximum progress ≥ 0.60, then ≥ 0.85 and endpoint distance ≤ 0.055 m.	420 21 s
t3_full	Acquire the wand with bilateral contact for 0.05 s, then complete the phase stages above. In a later sample, retain current progress ≥ 0.85 and move at least 0.045 m from both path and endpoint. Finally open (< -0.5) with no fingerpad contact. Releasing before exit fails.	640 32 s
t4_sphere	Approach within 0.10 m; first qualifying contact after ramp exit; any tray contact before success fails. Bilateral hold for nominal 25 ms (12 physics samples, 24 ms). Raise COM by 0.05 m from its height when the bilateral-hold criterion is first met, and retain contact and height for 0.50 s.	120 6 s
t4_sphere_tabletop; t4_slider	Approach within 0.10 m; contact after ramp exit, while supported, in the capture region, and moving at planar speed ≥ 0.08 m/s. Bilateral hold for 0.05 s, COM lift ≥ 0.05 m from the first attainment of that hold criterion, then contact and height for 0.50 s.	120 6 s
t4_can	Same supported-capture sequence. Before first fingerpad contact, a can-axis absolute dot product with vertical > 0.45 fails as unstable tipping. The configuration does not require a transverse closing axis.	120 6 s
t5_replan	Exactly one selected-side impact group completes before robot contact. At 30 ms after the physical impact-end timestamp, branch-signed lateral speed ≥ 0.035 m/s and yaw rate ≥ 0.35 rad/s. Approach within 0.10 m; bilateral hold with grasp-axis error $\leq 18^\circ$ for 0.05 s; COM lift ≥ 0.05 m from the first attainment of that hold criterion and stable hold for 0.50 s. Impact-group lag is 12 ms and maximum contact is 0.10 s.	120 6 s

context-active group contains the tasks with an obstacle clearance or a moving context (`t1_belt`, `t1_occ`, `t3_phase`, `t3_full`, and the four capture tasks and `t5_replan`).

Terminal events. We check success at every 2 ms physics step, after the ordered requirements of Table S2, and an interrupted continuous condition resets its counter. Terminal events fall into five categories. *Invalid* events are corrupted simulator states. *Boundary* events are a workspace or downstream exit, a drop, or an expired goal window. *Safety* events are unsafe contact, an excessive rim impulse, wire contact, a release impulse, early robot contact, unsupported tray contact, or a striker trap. *Task violations* are a release outside the goal, a path deviation or backtracking, a premature release, a deflection without capture, tipping, a missing or extra impact, or an insufficient change of outgoing motion. The last category is the *timeout*. Each family evaluates its events in a fixed order per physics step and keeps the first terminal event.

B.2. Scenarios, Demonstrations, and Splits

We sample continuous ranges uniformly and discrete choices with equal probability, and then apply physical-feasibility checks. Evaluation replays the stored accepted scenarios. Harmonic translation follows $\mathbf{p}(t) = \mathbf{c} + (0, v_y, 0)t + (A_x \sin(2\pi f_x t + \phi), 0, 0)$ with $\phi \sim \mathcal{U}[-\pi, \pi]$. Capture targets start at rest and move under gravity and friction on ramps of $[5, 15]^\circ$. We accept an occluded scene only if the target is visible for at least eight initial frames, fully occluded for 0.5–3.0 s, and visible again for at least three frames.

The training pool contains 11,200 demonstrations, 800 per task, generated by experts with privileged simulator state;

Table S3. **Roles and tolerances of the Dynamic-14 tasks.** The target is the body whose pose error enters a contact or containment tolerance ε. The context comprises obstacles, avoided with clearance m, and drivers that move the target. Speeds follow the sampling ranges of Table S4 (capture tasks: at the ramp exit). Terms: error terms that a task exercises (Eq. 2 and Appendix A.1). v: moving target. $r\theta$: orientation enters the contact. e_{dyn}: non-constant motion (harmonic, gravity, friction, impact). m: a context body must be avoided. Occ.: history gaps. Delay: command delay built into the task.

Task	Code	Target	Context	ε (mm)	$\|\mathbf{v}\|$ (m/s)	m (mm)	Terms
Harmonic grasp	grasp	block	table	[X]	0.03–0.20	–	v, $r\theta$, e_{dyn}
Conveyor grasp	t1_belt	part	belt	[X]	0.04–0.10	–	v
Translating grasp	t1_xyz	part	table	[X]	0.04–0.15	–	v, e_{dyn}
Rotating grasp	t1_so3	part ($\omega \leq 1.2$ rad/s)	table	[X]	0.04–0.15	–	v, $r\theta$, e_{dyn}
Occluded grasp	t1_occ	part	occluder canopy	[X]	0.04–0.10	[X]	v, m, occ.
Moving receptacle	t2_trans	receptacle	table	[X]	0.03–0.06	–	v
Rotating receptacle	t2_se3	T-shaped receptacle	table	[X]	0.03–0.06	–	v, $r\theta$
Wire passage	t3_phase	wire loop (entry)	rotating wire	30	[X]	swept	v, $r\theta$, m
Wire passage, full	t3_full	wire loop (entry)	rotating wire	30	[X]	swept	v, $r\theta$, m
Ball capture	t4_sphere	ball	tray, ramp	[X]	[0.4–0.7]	[X]	v, e_{dyn}, m
Ball capture, table	t4_sphere_tabletop	ball	table, ramp	[X]	[0.4–0.7]	–	v, e_{dyn}
Slider capture	t4_slider	slider	table, ramp	[X]	[0.4–0.8]	–	v, e_{dyn}
Can capture	t4_can	can	table, ramp	[X]	[0.4–0.7]	–	v, $r\theta$, e_{dyn}
Post-impact grasp	t5_replan	part	striker	[X]	0.12–0.14	[X]	v, $r\theta$, e_{dyn}, m, delay

the learned policies receive only the inputs of Sec. 3. We record the demonstrations at 20 Hz with camera images, depth, robot states, issued commands, and simulator ground truth for target poses and scene geometry, from which we compute the future labels, their validity masks, and the command labels.

We select configurations on 1,400 development scenes (100 per task). All Dynamic-14 results use a separate held-out set of 1,400 scenes generated with the same ranges and new source seeds that appear in no training, validation, or development episode; the first 100 feasible scenes in increasing seed order form each task's set.

B.3. Execution Protocol and Latency Measurement

Executor. Every method runs under the executor of Sec. 3.4 on a 50 ms control clock, with queries starting on its period boundaries. The total latency L_t of query t is the time that physics advances while it runs: its inference time under the protocol plus any added delay. Command j of its chunk executes at $\tau^{\text{a}}_{t,j}$ of Eq. 1, with $d_{\text{task}} = 50$ ms for the post-impact grasp and zero otherwise. Its commands $j < k_t$, with $k_t = \lceil L_t/\Delta_a \rceil$, have expired and are discarded, so query t executes commands $j = k_t, \ldots, \min(8, k_t + k_{t+1}) - 1$, one per control period. When $k_t + k_{t+1} > 8$, e.g., for a constant total latency above 200 ms, a chunk runs out before the next one arrives, and if $k_t \geq 8$, all its commands have expired. Before the first chunk of an episode arrives, and whenever a chunk runs out, the controller holds the current arm target and gripper command while physics advances; no command is repeated. The reported time-to-action is the measured latency before rounding to $k_t\Delta_a$.

Real-time protocol. Unless noted otherwise, physics advances during inference by the method's native latency, drawn per query from its latency distribution measured at batch size 1 on one RTX 5090. The native-latency ACT/DP rollouts run on RTX 5090; ACT paused and added-latency rollouts and the planned remaining evaluations use RTX PRO 6000. Their own compute time does not enter the episode. Table S7 reports three alternatives. With *paused* physics, the executor is unchanged and the simulation waits for inference; an added delay still advances physics. Without added delay, $L_t = 0$, and command 0 executes at capture. With *equal latency*, every method is charged the p95 latency of SE(3)-WAM regardless of its own speed. With *original executors*, each baseline keeps the execution scheme of its paper, e.g., a longer chunk before replanning.

Added latency. An added latency of 0, 50, 100, 150, or 200 ms is inserted after inference on top of the protocol's latency, and physics advances during it.

Latency measurement. We time all methods on one RTX 5090 (32 GB), the GPU of our robot, with exclusive use and batch size 1. These measurements form the latency distributions that the simulated rollouts replay independently of the rollout hardware. For each method, each of three rounds uses thirty warm-up calls and 300 measured calls over three observation sequences of eight frames, with CUDA synchronization at call boundaries and a rotated method order. Each method uses the precision of its release; SE(3)-WAM uses FP32. The time-to-action includes encoding the image and generating the first command, and on the robot also tracking the newest frame. For SE(3)-WAM, we time the

Table S4. Scenario proposal ranges. Length, linear velocity, angle, and angular velocity use m, m/s, rad, and rad/s unless marked otherwise. Positions are in world coordinates except the stated ramp-local offsets. Continuous intervals are uniform proposals; discrete choices are equiprobable before feasibility checks. Evaluation replays the stored accepted scenarios, whose distribution need not remain uniform. Harmonic centers are not initial positions.

Task	Initial-state proposal	Motion or event parameters
grasp	Harmonic center $c_x \in [-.08, .08]$, $c_y \in [-.26, -.19]$, $z = .822$; yaw $[-1.1, 1.1]$.	$v_y \in [.025, .125]$, $A_x \in [.015, .035]$, $f_x \in [.3, .7]$ Hz; $\omega_z \in [-1.2, 1.2]$.
t1_belt	Target $x \in [-.08, .08]$, $y \in [-.26, -.18]$, $z = .822$; identity orientation, zero initial twist.	Belt $v_y \in [.04, .10]$.
t1_occ	Same target proposal as Belt; canopy at $(.322, -.29, 1.20)$.	Same belt range; clearance parameter .20; physical occlusion.
t1_xyz	Harmonic center $c_x \in [-.025, .025]$, $c_y \in [-.24, -.20]$, $z = .818$; yaw $[-.35, .35]$.	$v_y \in [.04, .09]$, $A_x \in [0, .03]$, $f_x \in [.25, .65]$ Hz; $\omega_z = 0$.
t1_so3	Same as XYZ.	Same translation; $\omega_z \in [-1.2, -.4] \cup [.4, 1.2]$, equal sign probabilities.
t2_trans	Goal $x \in [-.13, -.08]$, $y \in [-.18, -.12]$, $z = .825$; fixed orientation.	Goal $v_y \in [.03, .06]$; no rotation.
t2_se3	Same goal x, y range, $z = .804$; yaw $[-.3, .3]$.	Goal $v_y \in [.03, .06]$; $\omega_z \in [-.25, -.08] \cup [.08, .25]$, equal sign probabilities.
t3_phase	Wand $x, y \in [-.03, .03]$, $z = .870$, yaw $[0, 2\pi]$; wire phase $[-.25, .35]$.	Wire center $(.10, -.02, .94)$; world-X rotation $\omega_x = -\{2.5, 3, \ldots, 7.5\}^\circ$/s (11 equal-probability values).
t3_full	Wand at $(-.05, -.02, 1.04)$ with identity orientation; same wire phase range.	Same wire rotation; cradle and release requirement enabled.
t4_sphere	Ramp-local offsets: lateral $[-.015, .015]$, longitudinal/normal $(-.145, .030)$; lip $(.06, -.02, 1.10)$.	Gravity-driven from rest; gate release at tick 0; slope $[5, 15]^\circ$; target/ramp sliding friction $[.7, .9]$; unsupported capture.
t4_sphere_tabletop	Lateral $[-.015, .015]$, longitudinal/normal $(-.124, .034)$; lip $(0, -.06, .808)$.	Same slope and release; target/ramp friction $[.7, .9]$, table friction .35.
t4_slider	Lateral $[-.015, .015]$, longitudinal/normal $(-.129, .024)$; supported ramp.	Same slope and release; target/ramp friction $[.025, .045]$, table friction $[.012, .018]$.
t4_can	Lateral $[-.010, .010]$, ramp-axis yaw $[-4, 4]^\circ$; longitudinal position follows gate geometry and oriented can extent.	Same slope and release; target/ramp friction $[.25, .45]$, table friction $[.09, .15]$.
t5_replan	$x \in [-.005, .005]$, $y = -.29$, $z = .812$; yaw $[-.08, .08]$; zero initial twist.	Belt $v_y \in [.12, .14]$; equal-probability left/right striker triggered at $y = -.13$; stroke .098, pulse .35 s, actuator scale $[1, 1.1]$, force cap 4 N; command delay 50 ms.

forecast heads separately, although deployment skips them. We distinguish three times. The *time-to-action* is the measured wall-clock time from the arrival of an observation to the availability of its first command, without rounding; the *handover time* $\ell_t = k_t \Delta_a$ rounds the total latency up to the control period; and the *action age* is the execution time of a command minus the capture time. On the robot, traces record capture, availability, preprocessing, the start and end of inference, queue entry, command issue, and execution. A query misses its deadline when its time-to-action exceeds the 50 ms control period, so that the next period executes a remaining command or holds. Appendix C.2 reports the measured values.

B.4. Baselines and Ablation Controls

Common conditions. Table S5 summarizes the adaptations. Each baseline keeps its official architecture, action chunk, and initialization. ACT and Diffusion Policy train from scratch, $\pi_{0.5}$ is fine-tuned from `pi05_base`, and the other methods start from their released weights. All of them train on the same 11,200 demonstrations, whose commands we map to each action space through the same Cartesian controller. The seven trained baseline endpoints use agent-view RGB only, as does SE(3)-WAM. The native missing-view paths are retained: $\pi_{0.5}$ masks its two zero wrist slots; DreamVLA uses a preprocessed black wrist image with wrist auxiliary supervision masked; OpenWAM-α and DynamicWAM fill their missing wrist regions with black. Each baseline follows its official update budget, trained once with

Table S5. **Baseline adaptations.** All methods train on the same 11,200 demonstrations with the inputs listed below and run under the shared executor of Appendix B.3. Updates × batch follows the official recipe. Exec.: the execution scheme of the original paper (commands executed per chunk before replanning), used only for the original-executor control of Table S7. Params: trainable parameters, frozen encoders excluded. Seeds: training seeds.

Method	Initialization	Inputs	Updates × batch	Exec.	Params	Seeds
ACT [66]	scratch	current agent RGB, robot state	50,000 × 32	temporal agg.	[X]	1
Diffusion Policy [10]	scratch	two causal agent RGB frames, robot state	50,000 × 32	8	[X]	1
$\pi_{0.5}$ [35]	`pi05_base`	current agent RGB, robot state, task text; wrist slots masked	30,000 × 256	10	[X]	1
$\pi_{0.5}$ + SE(3)	`pi05_base`	planned: $\pi_{0.5}$ + SE(3) tokens; camera inputs pending	30,000 × 256	10	[X]	1
DynamicVLA [54]	released DOM ckpt.	frames $[-2, 0]$, robot state, task text	50,000 × 44	streaming	[X]	1
PUMA [12]	official checkpoint	RGB, optical-flow history, robot state, task text	[X]×[X]	[X]	[X]	1
DreamVLA [63]	released components	seven causal agent RGB frames, robot state, task text; black wrist input	48,220 × 512	1	[X]	1
OpenWAM-α [49]	official foundation	current agent RGB, joints, gripper, task text; missing views black	34,490 × 192	32	[X]	1
DynamicWAM [31]	official initialization	RGB, flow history, kinematic descriptors	40,000 × 128	16	[X]	1
DynamicWAM + SE(3)	official initialization	planned: DynamicWAM + SE(3) tokens; camera inputs pending	40,000 × 128	16	[X]	1
SE(3)-WAM	frozen MAE encoder	current agent RGB-D, SE(3) and robot histories	51,740 × 512	1	523.8M	3

seed 0; the input and action adaptations are stated here and in Table S5. For DreamVLA, we ran both its original 40-epoch schedule and a 20-epoch cosine schedule and report the one that was better on development scenes.

Inputs. PUMA [12] receives its historical optical flow, computed on the rendered frames. DreamVLA receives seven causal RGB frames, the end-effector position and orientation, the finger positions, and a fixed task text, and it samples with ten DDIM steps and classifier-free guidance of 1.5. OpenWAM-α receives the current RGB views, joint values, gripper opening, and task text, and it generates video and actions jointly with video decoding disabled. DynamicWAM receives its flow history and kinematic descriptors.

SE(3)-augmented baselines. $\pi_{0.5}$+SE(3) and DynamicWAM+SE(3) receive the eight target-state tokens of Appendix A.2 from the same pose source, with the same validity flags and masking. A two-layer MLP maps the tokens to the model's width. For $\pi_{0.5}$, we append them to the prefix after the image tokens, where the action expert attends to them; for DynamicWAM, they join its kinematic descriptors. The output layer of the MLP is zero-initialized, and training follows the base model's recipe.

Ablation controls. The five controls of Sec. 4.3 change one component each of the configuration of Table S1 and keep the data, the recipe, the update budget, the executor, and the training seeds 0, 1, and 2. *No target history* removes slots 0–6 of the target-state tokens, which carry the past poses with their velocity and acceleration features, and keeps slot 7 with the current pose (Appendix A.2). *No future supervision* removes $\mathcal{L}_P$, $\mathcal{L}_F$, and the three future appearance losses with their heads, and keeps the reconstruction of the current target region. *No scene-flow supervision* removes $\mathcal{L}_F$ and the flow readout, and *no dedicated SE(3)* removes $\mathcal{L}_P$ and the pose head; both keep the target-state tokens and all appearance targets. The *uniform-fidelity forecast* replaces $\mathcal{L}_P$ and $\mathcal{L}_F$ by [a forecast of future latents of a frozen video VAE of the whole frame at the same horizons, its readout, and its loss weight]. Since every forecast head is skipped at deployment, only the no-history control changes the inputs at inference.

B.5. Metrics and Statistics

The task-balanced success rate is

$$\mathrm{SR}(m) = \frac{100}{14} \sum_{k=1}^{14} \frac{1}{S_m n_k} \sum_{s=1}^{S_m} \sum_{i=1}^{n_k} y_{mski}, \qquad \text{(S4)}$$

for binary outcomes y_{mski} of method m, trained with seed s, on scene i of task k, with $S_m = 3$ training seeds for our variants and $S_m = 1$ for the baselines. The *position error at first contact* e_p is the distance of the end effector from the expert's contact frame on the target at the first contact. The two placement tasks start with the object held, so their contact is the first contact of the held object with the receptacle, and their contact frame is the end-effector pose implied by the expert's placement target. The phase-only passage (`t3_phase`) succeeds without any contact, so e_p averages over the other 13 tasks; we report it on episodes that make contact, and failures without contact stay in the SR denominator.

Training seeds and intervals. The full model and the five ablation controls use training seeds 0, 1, and 2, and each

baseline uses seed 0. On development scenes, three training seeds of the same configuration differed by up to [7.4] points in SR, mostly on the wire passage, whose success ranged from [0] to [71]% across seeds. We report each training seed and the cross-seed mean and standard deviation (Appendix C.4); for the ablations, we also repeat the comparison without the phase-only wire passage (`t3_phase`). For paired 95% intervals, we average the seeds per scene and bootstrap held-out scenes within each task, jointly across methods. These intervals quantify scene uncertainty for the trained weights, and we treat differences whose interval lies within ± 2 points as ties.

C. Additional Simulation Results

C.1. Per-Task Results

Table S6 breaks the held-out Dynamic-14 results of Table 1 down by task for every method, with the paired interval of SE(3)-WAM against the strongest other method on each task; its last two rows give the position error at first contact e_p and the number of episodes with a contact (Appendix B.5). With its native 100-step DDPM sampler, every chunk of Diffusion Policy has expired when it arrives, so none of its commands is executed. [Tasks on which every method fails or a baseline outperforms SE(3)-WAM, and the dominant failure of SE(3)-WAM.]

C.2. Evaluation Protocols and Latency

Table S7 evaluates every method under three further protocols. Paused physics shows how the relative performance changes when the cost of computation is ignored, equal latency compares all methods at the p95 latency of SE(3)-WAM, and the original executors check whether the shared executor removes a capability that a baseline relies on. With paused physics, the difference between SE(3)-WAM and [name] is [$\pm$X.X] points, and under equal latency it is [$\pm$X.X] points. The original executors change the SR of the baselines by at most [X.X] points. Table S8 gives the success at every added latency; its rows for SE(3)-WAM and the strongest baseline are the data of Fig. 1 (right).

Table S7. **Protocol controls** on held-out Dynamic-14 scenes (SR, %). Paused: shared executor, with physics stopped during inference. Equal latency: shared executor, with every method charged the p95 latency of SE(3)-WAM. Original executor: real time with each method's own execution scheme. Real time: the protocol of Table 1. Late: queries in real time whose time-to-action exceeds the 50 ms control period (%). Age: p95 action age in real time (ms); –: no command executed.

Method	Paused	Equal latency	Original executor	Real time	Late (%)	Age p95
ACT	40.9	—	—	42.0	0.0	50
Diffusion Policy	—	—	—	0.0	100.0	–
$\pi_{0.5}$	—	—	—	—	—	—
$\pi_{0.5}$ + SE(3)	—	—	—	—	—	—
DynamicVLA	—	—	—	—	—	—
PUMA	—	—	—	—	—	—
DreamVLA	—	—	—	—	—	—
OpenWAM-α	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—
DynamicWAM + SE(3)	—	—	—	—	—	—
SE(3)-WAM	—	—	–	—	—	—

Table S8. **Success versus added latency** on held-out Dynamic-14 scenes (SR in %, real-time protocol). The 0 and 150 ms columns equal the Native and +150 ms columns of Table 1.

	Added latency (ms)				
Method	0	50	100	150	200
ACT	42.0	—	—	24.1	—
Diffusion Policy	0.0	—	—	0.0	—
$\pi_{0.5}$	—	—	—	—	—
$\pi_{0.5}$ + SE(3)	—	—	—	—	—
DynamicVLA	—	—	—	—	—
PUMA	—	—	—	—	—
DreamVLA	—	—	—	—	—
OpenWAM-α	—	—	—	—	—
DynamicWAM	—	—	—	—	—
DynamicWAM + SE(3)	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—

The native-latency ACT/DP rollouts ran on RTX 5090; ACT paused and ACT/DP added-latency rollouts ran on RTX PRO 6000. Each method reuses its recorded RTX 5090 latency distribution.

Timing. Table S9 breaks the time-to-action of SE(3)-WAM on one RTX 5090 down by stage, measured as in Appendix B.3; Table 1 gives the p95 latency of every method and Table S5 the parameter counts. On the robot, we report the p50, p95, and maximum action age and the fraction of all queries of the test trials that miss the 50 ms deadline, together with the peak memory.

Table S6. **Per-task held-out success** (%) on Dynamic-14 for every method of Table 1, mean of three training seeds for our variants and one for baselines, real-time protocol. DP: Diffusion Policy. DVLA: DynamicVLA. DrVLA: DreamVLA. OWAM: OpenWAM-α. DWAM: DynamicWAM. +: with our SE(3) history tokens. σ: SE(3)-WAM with nominal synthetic correlated pose noise (Appendix C.5). Δ: SE(3)-WAM minus the strongest other method on that task, with its 95% paired interval. The last two rows give the position error at first contact e_p (mm), averaged over the 13 tasks with a contact, and the number n of episodes with a contact (Appendix B.5); –: no 13-task mean. Task codes are listed in Table S3.

Task	ACT	DP	$\pi_{0.5}$	$\pi_{0.5}$+	DVLA	PUMA	DrVLA	OWAM	DWAM	DWAM+	σ	**Ours**	Δ [95% CI]
Harmonic grasp	48.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Conveyor grasp	46.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Translating grasp	18.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Rotating grasp	21.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Occluded grasp	32.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Moving receptacle	26.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Rotating receptacle	0.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Wire passage	99.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Wire passage, full	0.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Ball capture	36.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Ball capture, table	71.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Slider capture	57.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Can capture	90.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Post-impact grasp	44.0	0.0	—	—	—	—	—	—	—	—	—	—	—
Task-balanced	42.0	0.0	—	—	—	—	—	—	—	—	—	—	—
e_p (mm)	32.2	–	—	—	—	—	—	—	—	—	—	—	–
n	1109	6	—	—	—	—	—	—	—	—	—	—	–

Table S9. **Latency breakdown** of one query of SE(3)-WAM on one RTX 5090 at batch size 1 (ms, p50 / p95), and action age on the robot.

Stage	p50	p95
Pose tracking (per frame, robot)	—	—
Image encoding	—	—
World stream (once per query)	—	—
Action stream (5 Euler steps)	—	—
Time-to-action (total)	—	—
Forecast heads (training only, for reference)	—	—
Action age at execution (robot)	—	—
Queries over 50 ms (robot, %)	—	

C.3. Results on DOMINO

On DOMINO [12], we compare SE(3)-WAM, $\pi_{0.5}$, DynamicWAM, and PUMA on 24 single-arm Franka tasks of its clean level-1 setting. All methods use the benchmark's demonstrations, scenes, and native evaluator, one training seed, and 50 episodes per task and method. We report SR and the manipulation score (MS), which scales route completion by penalties for leaving the workspace or view and for collisions (Table 2). SE(3)-WAM uses domain-specific normalization and reads the simulator pose of the target named by the instruction; slots without a valid pose are masked as in Appendix A.2. The RGB-based baselines do not receive this pose. [Summarize the DOMINO results.]

C.4. Final Ablations and Seed Stability

Table S10 reports the five controls of Sec. 4.3 (Appendix B.4) for each training seed, with the cross-seed mean and standard deviation and the paired difference from the full model, with native latency and with 150 ms of added latency. The last two columns repeat the paired differences without the phase-only wire passage (`t3_phase`), whose success varied most across training seeds (Appendix B.5). The two task groups of Fig. 5 are listed in Appendix B.1. [Whether any conclusion of Sec. 4.3 changes for an individual seed or without `t3_phase`.]

C.5. Robustness to Pose Noise

With frozen weights, we evaluate three synthetic perturbations of the target pose: nominal temporally correlated noise, twice its amplitude, and nominal noise with contiguous missing observations. The nominal process combines a fixed per-episode bias and a stationary correlated component, each with per-axis standard deviations of 5 mm and 1°, with a correlation time of 200 ms. Dropouts last 200 ms, separated by geometrically distributed valid gaps with a mean of 3.8 s, giving a long-run missing fraction of 5%. The perturbation applies to the current and past estimates before the history features are computed.

Fig. S1 evaluates the final model under these perturbations, on the same held-out scenes at zero added latency and

Table S10. **Final ablations by training seed** on held-out Dynamic-14 scenes (Appendix C.4). S0–S2: SR (%) of training seeds 0–2; mean ± s.d. over seeds; Δ: control minus SE(3)-WAM (points) with its paired 95% interval. Last two columns: Δ without the phase-only wire passage (`t3_phase`), 13 tasks.

	Native					+150 ms					Δ w/o `t3_phase`	
Variant	S0	S1	S2	Mean ± s.d.	Δ [95%]	S0	S1	S2	Mean ± s.d.	Δ [95%]	Native	+150 ms
SE(3)-WAM *	—	—	—	—	–	—	—	—	—	–	–	–
No target history	—	—	—	—	—	—	—	—	—	—	—	—
No future supervision	—	—	—	—	—	—	—	—	—	—	—	—
No scene-flow supervision	—	—	—	—	—	—	—	—	—	—	—	—
No dedicated SE(3)	—	—	—	—	—	—	—	—	—	—	—	—
Uniform-fidelity forecast	—	—	—	—	—	—	—	—	—	—	—	—

with all three training seeds. Relative to clean poses, SR changes by [X.X], [X.X], and [X.X] points under nominal correlated noise, twice its amplitude, and missing observations.

Figure placeholder: Fig. S1 — pose noise
x: clean poses, nominal correlated noise, twice its amplitude, and nominal noise with missing observations; y: SR of SE(3)-WAM per task family, frozen weights, real-time protocol; paired intervals over matched scenes.

Figure S1. **Robustness to pose noise.** SR of SE(3)-WAM under synthetic perturbations of the target pose, relative to clean poses.

C.6. Forecast Diagnostics

Table S11 compares the learned SE(3) forecast of the final model offline with constant-velocity and constant-acceleration extrapolation from the same pose history.

Table S11. **Offline prediction errors.** Target-position error at 400 ms (mm) on the same held-out observations. CV and CA are analytic extrapolations from causal pose history; the learned head reads frozen full-model states.

	Const. vel.	Harmonic	Roll/slide	Impact
CV extrapolation	—	—	—	—
CA extrapolation	—	—	—	—
Learned pose head	—	—	—	—

D. Real-Robot Experiments

D.1. Setup and Tasks

The robot is a Franka arm ([model]) with [gripper model] and a fixed RealSense D435 RGB-D camera, commanded through [a timestamped Cartesian interface] at [control frequency]. The camera records RGB-D at [resolution] and [frame rate], and we calibrate its extrinsics by [procedure] before each session. SE(3)-WAM uses this fixed third-person view; baselines whose recipes require a wrist view receive [per-method camera configuration]. All policies run on one RTX 5090. Fig. S2 shows the layout, and Table S12 defines the six tasks. T1 and T2 share a belt conveyor, T3 uses a turntable, T4 a remotely driven toy car, T5 a cue-driven collision between two balls, and T6 a ramp. Touching or pushing the target without completing the task, a single-finger hold, and an action after the target has left the workspace count as failures; [equipment-invalid trials and safety interventions are repeated or counted as failures by the following rule].

D.2. Pose Tracking and Demonstration Labels

Tracking is marker-free. Grounding DINO [28] localizes the object named by the task in the first frame, SAM 2.1 [36] gives its initial mask, and FoundationPose [50] initializes the object's 6D pose from this mask and then tracks it. [FoundationPose variant and mode (model-based with an object mesh or model-free with reference views), how the object frame is kept consistent across demonstrations and trials, and re-initialization after a loss.] The tracker runs on the control clock: at every control-period boundary, it updates the pose on the newest camera frame captured at or before the boundary, so the pose history has the 50 ms spacing used in training, and each estimate keeps the capture time of its frame. A query waits for the update of its own boundary and then runs the policy on the same GPU. While a query spans further boundaries, the tracker keeps updating at a higher GPU priority and processes only the newest frame, so that an update never queues behind older ones; the tracking time and this contention are part of the measured latency (Table S9). [Camera frame rate and the measured time per update.] When the target is lost, the estimate is marked invalid, and the previous pose is not carried forward. [How the target poses of the demonstrations, used as inputs and training labels, are obtained.] For the ball tasks, the target-state tokens keep the translation, velocity, and acceleration of the ball center and mask its orientation and angular motion, and the pose head is supervised on translation only. Table S13

Figure placeholder: Fig. S2 — real-robot setup

(a) Left, photo of the workcell from behind the robot: the third-person RGB-D camera, the robot, and the conveyor, turntable, car track, ball-collision track, and ramp at their operating positions, each labeled; the wrist camera if a baseline uses one. **(b) Right, top-down schematic to scale:** the robot base frame with its axes, the camera position and viewing direction with its distance to the workspace, the footprint and motion direction of each device for T1–T6, the grasp or interception region of each task, and the workspace bounds in cm. **Rules:** real photo, unretouched; schematic from the calibrated extrinsics and measured device positions; no repetition of the task photos of Fig. 6.

Figure S2. **Real-robot setup.** (a) The workcell. (b) Top-down layout of the camera, the robot base frame, and the devices of T1–T6 [with the workspace dimensions].

Table S12. Real-robot task definitions. $N = 100$: training demonstrations per task; 20 additional demonstrations per task are reserved for validation.

Task	Device, motion, and objects	Operating range; N	Success and failure criteria
T1	Conveyor transports a graspable cube.	[belt speed]; 100	[stable grasp and lift; failures]
T2	Conveyor transports an open box; the arm places a cube into it.	[belt speed]; 100	[placement and retention; failures]
T3	Turntable carries a cube around its axis.	[angular speed, radius]; 100	[stable grasp and lift; failures]
T4	Remotely driven toy car moves on the tabletop.	[speed, path]; 100	[car grasp and lift; failures]
T5	A cue-driven ball strikes a second ball; the arm intercepts the second ball after the collision.	[impact and outgoing speed]; 100	[post-impact capture; failures]
T6	A ball rolls or a block slides down a ramp toward the interception region.	[slope, release, speed]; 100	[capture and retention; failures]

Table S13. **Real-robot pose tracking.** Median/p95 translation (mm) and rotation (°) error of the tracker against a marker-free reference, the uncertainty of that reference, and the fraction of lost observations. Ball rotation is not used.

Target	mm	°	Ref. uncertainty	Lost
Real rigid object	—	—	—	—
Real ball	—	–	—	—

reports the tracking accuracy.

D.3. Training and Trial Protocol

We compare SE(3)-WAM, $\pi_{0.5}$, DynamicWAM, and Diffusion Policy [10], with 20 trials per task and method. All methods use the same pre-generated trial keys, which fix the object instance, the initial pose, and the motion parameters, sampled by [sampling protocol]. Trials run in [randomized method order], so that drift of the apparatus does not align with the methods. An operator resets the scene. A trial starts at the first usable observation, the robot may move at any time, and a trial ends at success, failure, or a timeout of [X] s. Demonstrations, development trials, and test trials come from separate sessions. SE(3)-WAM is fine-tuned per task from the DROID base (Appendix A.6), and the baselines are trained or fine-tuned on the same demonstrations with [per-method initialization and training settings]. For every method, we log the end-to-end time-to-action of each query on the same RTX 5090, including tracking for SE(3)-WAM; Table 4 reports its p95.

D.4. Detailed Results and Failures

Table S14 reports successes per task, 95% binomial intervals for the pooled rates, and paired differences from a bootstrap over trial keys. [The most frequent failures of SE(3)-WAM and their causes.]

Table S14. **Real-robot successes** per task (successes/trials, 20 trials per task; tasks in Table S12). All: pooled success rate (%) with its 95% binomial interval. Δ: paired difference from SE(3)-WAM (points) with a bootstrap interval over trial keys.

Method	T1	T2	T3	T4	T5	T6	All	Δ
$\pi_{0.5}$	—	—	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—	—	—
Diffusion Policy	—	—	—	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—	—	—	–

E. Preliminary Design Studies

We screened the encoder, the history representation, the forecast targets, and the coupling of world and action streams with smaller models: 4 layers of width 768, trained for 4 epochs (40,104 updates at a global batch size of 16) and evaluated with the synchronous executor that preceded our real-time protocol. Each group keeps its own encoder and seed count, stated below, and conditions are compared within a group. These studies selected the components of the final model; the five controls of Sec. 4.3 then test its central components at full scale and under the real-time protocol (Appendix C.4).

E.1. Encoder, Backbone, and History

Encoder and backbone. We compared a CNN trained with the policy, frozen DINOv3 ViT-S/16 and ViT-L/16 [40], and MAE ViT-B/16 [16] (three seeds for the MAE reference; Table S15), and selected MAE. We also compared a backbone with shared attention and stream-specific feed-forward networks against one with fully separate world and action parameters, at about 47.3M backbone parameters and with three seeds each ([source of the backbone comparison]), and selected the shared attention.

History. With a frozen DINO-L encoder and one seed per condition, we compared the current state alone, the target-motion history, RGB windows, Farnebäck flow [13], flow with motion statistics [31], codec motion vectors, video-VAE latents, and compressed windows (Table S16). Its last row replaces the past target poses of the trained history model by its current pose at test time.

E.2. Forecast Targets and Auxiliary Supervision

With a frozen DINO-L encoder and one seed per condition, we compared fifteen future targets, from target-only and image-space forecasts to role-aware combinations (Table S17). A second group with the MAE encoder and one seed removed each future appearance target from the selected configuration. In the first group, no future target improved clearly on the variant without future supervision (P00, 62.78% against 62.38% for scene flow and SE(3)); single seeds at this scale do not resolve such differences, and the full-scale ablations of Sec. 4.3 test future supervision in the final model.

Table S15. **Observation encoders** at the screening scale (Appendix E.1). Params: encoder parameters; p95: time-to-action.

Encoder	Params	Trained	SR	SR_{150}	p95
CNN	—	yes	—	—	—
DINOv3 ViT-S/16	22M	frozen	—	—	—
DINOv3 ViT-L/16	300M	frozen	—	—	—
MAE ViT-B/16*	86M	frozen	—	—	—

Table S16. **History representations** at the screening scale (frozen DINO-L encoder, one training seed, 1,400 scenes). The first eight conditions receive the current target pose; the ninth adds the target-motion history. Last row: the ninth model with its past target poses replaced by the current pose at test time and the kinematic features recomputed.

History input	Success / 1,400	SR (%)
Current image	748	53.43
RGB window	662	47.29
Optical flow	720	51.43
Flow + motion statistics	723	51.64
Codec motion vectors	733	52.36
Video-VAE latent	710	50.71
Compressed window (age)	634	45.29
Compressed window (uniform)	670	47.86
Current image + target-motion history	824	58.86
stationary history at test time	9	0.64

E.3. World–Action Coupling

With the MAE encoder and three seeds per condition, we compared six couplings of the world and action streams. R0 disables every forecast loss. R1 adds one object query and one query per patch to the world stream, which attend to $\mathcal{S}$, $\mathcal{M}$, and each other and carry the forecast readouts, without any edge to the actions. R2 lets the action tokens attend to these queries at every layer, R3 lets the queries attend to the actions, and R4 opens both edges; in R3 and R4, the world states are recomputed at every Euler step. R5 decodes the forecast once per query and encodes the target trajectory and the flow with visibility, with rotations in a continuous 6D form, through a geometry encoder (0.37M parameters) into tokens that the actions attend to. Future supervision without

Table S17. **Forecast targets** at the screening scale. The first group uses a frozen DINO-L encoder and seed 0, with its 140 development scenes and the 1,260 remaining scenes reported separately. The second group uses the MAE encoder and seed 1 on 1,400 scenes; it keeps scene flow, SE(3), and the current-target RGB reconstruction and removes future appearance targets. Compare conditions within each group. Targets are cumulative from the current state at the eight horizons, except codec motion: SE(3), the target pose; 3D scene flow, base-frame displacements of the 14×14 patch centers; 2D tracks and dense 2D displacement, image displacements of target points, grid points, or all pixels; codec motion, the motion vectors of a video codec; video-VAE latent, latents of a frozen Wan2.2 VAE; object DINO, mask-pooled DINOv3 features of the target.

ID	Future supervision	Dev. success / 140	Success / 1,260	SR_{1260} (%)
P00	None	88	791	62.78
P01	3D scene flow	95	782	62.06
P02	SE(3)	92	789	62.62
P03	Scene flow + SE(3)	93	786	62.38
P04	Video-VAE latent + SE(3)	92	783	62.14
P05	Dense 2D displacement + SE(3)	87	746	59.21
P06	Codec motion + SE(3)	86	795	63.10
P07	Sparse scene 2D tracks + SE(3)	90	807	64.05
P08	Scene flow + object DINO	89	794	63.02
P09	Scene flow + target 2D tracks	88	808	64.13
A_α	P03 + motion magnitude	89	778	61.75
P12	P03 + depth	90	791	62.78
P13	P03 + DINO	92	788	62.54
P14	P03 + dynamic RGB	89	790	62.70
P15	P03 + depth / DINO / dynamic RGB	94	802	63.65
AUX0 MAE-B: future appearance auxiliaries		Success / 1,400		SR_{1400} (%)
None		894		63.86
Depth only		867		61.93
DINO only		946		67.57
Dynamic RGB only		926		66.14
All three (existing reference)		995		71.07

Table S18. **Couplings R0–R5** at the screening scale (MAE encoder, three training seeds each, 1,400 development scenes per seed; Appendix E.3). SR: mean over seeds (%); s.d.: standard deviation over seeds (points).

Coupling	SR	s.d.
R0 (future losses off)	60.45	3.57
R1 (queries, no edge)	63.60	2.34
R2 (world→action)	62.52	1.97
R3 (action→world)	64.21	2.13
R4 (bidirectional)	63.00	4.19
R5 (decoded geometry)	63.95	1.91

any edge to the actions (R1) exceeded R0 by 3.14 points on average, with seed-wise differences of −1.29, +9.64, and +1.07. Feeding the forecasts to the actions gave no consistent further gain: R2 and R5 differed from R1 by −1.07 and +0.36 points on average, with standard deviations of 4.28 and 3.45 points over the seed-wise differences, and the effects of both cross-stream edges changed sign across seeds (Table S18). We therefore use the forecasts only as training supervision, which also keeps deployment free of their cost.